

DYNAMIC WEIGHT GRAFTING: LOCALIZING FINE-TUNED FACTUAL KNOWLEDGE IN TRANSFORMERS

Anonymous authors
Anonymous submission

ABSTRACT

When a language model acquires a new fact during finetuning—a newly released movie, a newly elected pope—where does that information come to reside, and through what internal route is it retrieved at generation time? Is the entity enriched with its associated relation as soon as it is processed, is the relation instead reconstructed just-in-time immediately before prediction, or do models maintain several redundant routes at once? Prevailing localization techniques, and activation patching in particular, are structurally ill-suited to adjudicating between these hypotheses: because they *replace* portions of the residual stream, they overwrite the very upstream computation whose contribution they set out to measure. We address this gap with **dynamic weight grafting**, an analysis technique that transplants *selected subsets of weights* from a finetuned model into a pretrained model at chosen components and token positions while leaving the surrounding computation to be recomputed intact. Applying the method to finetuned single-hop relation completion across four decoder-only Transformer families, we isolate two distinct pathways through which finetuned relation knowledge is retrieved during generation: an *enrichment* pathway that writes relation information into the residual stream while the model processes the entity tokens (e.g., “Zendaya” in “Zendaya co-starred with Timothée Chalamet”), and a *recall* pathway that reconstructs this information at the final token position immediately before the target fact is generated. Both pathways are required for some relations, whereas for others enrichment or recall alone suffices; jointly, the two pathways recover nearly the full finetuned model’s performance, while grafting everything *except* them drives accuracy to near zero. Within the family of grafting decompositions we test, these two positions are therefore together both sufficient and necessary for relation completion. Component-level grafting further localizes the recall pathway, attributing it jointly to task-specific attention operating on the entity and final-token positions and to an entity-specific extraction step in the feedforward networks of the final layers. Because dynamic weight grafting intervenes on parameters and components rather than on activations alone, it resolves *which* components carry finetuned knowledge at a granularity that activation-based interventions cannot reach.

1 INTRODUCTION

When a new pope is elected and we finetune a large language model (LLM) so that it answers “Who’s the pope?” correctly, how does the model come to implement this behavior? LLMs demonstrably store and retrieve an enormous number of the relationships and associations they acquire during pretraining (Petrone et al., 2019; Roberts et al., 2020); the complementary question—how a model encodes a *newly finetuned* fact in its parameters, and which internal mechanisms extract that fact during text generation—has received comparatively little attention. We take up precisely this question, and answer it at the resolution of individual model components and token positions.

Concretely, when new relational information is written into an LLM, is it deposited only into the entity representation (for instance, in the embeddings), is it “enriched” onto the entity token position in the lower and middle layers, or is it reconstructed in *response* to the entity in the higher layers, closer to next-token prediction? And, at finer granularity, which components retain newly learned relation information, and at which token positions do they contribute to its retrieval? Adjudicating among these possibilities demands an analysis that can attribute behavior to individual components

without disturbing the computation that feeds them—the one capability that existing activation-based methods structurally lack.

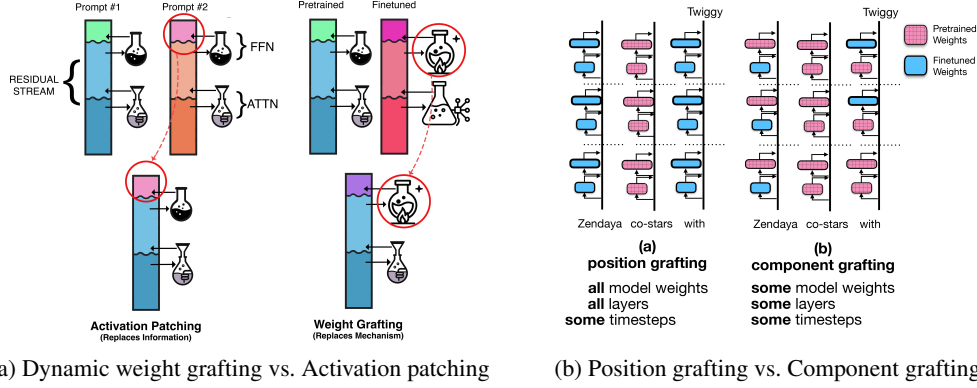


Figure 1: Dynamic weight grafting localizes the *components* underlying finetuned knowledge retrieval by transplanting *small subsets* of a supervised-finetuned (SFT) model’s weights into a pretrained model. (a) **Dynamic weight grafting versus activation patching.** Activation patching replaces the model’s activations at a chosen point with activations recorded from another run, thereby overwriting the upstream computation; dynamic weight grafting instead substitutes specific parameter matrices of the finetuned model into the pretrained model and lets the activations be recomputed by the grafted weights, leaving the surrounding computation intact. (b) **The grafting schemes used in our experiments.** *Position grafting* substitutes the entire pretrained or finetuned model—every layer and every component—at a given token position; *component grafting* blends pretrained and finetuned weights at the finer granularity of individual components, dynamically and per token position.

Our aim is to develop analysis tools that isolate which *components* invoke finetuned information: does the feedforward network on, say, the seventeenth layer at the final token position reconstruct the fact that Pope Leo XIV was recently elected, or was that information channeled forward from earlier hidden states by attention? Prior interpretability approaches to localizing relation knowledge rely predominantly on variants of *activation patching* (Meng et al., 2022a) or on ablations (Geva et al., 2023) to determine which components contribute to next-token prediction. These approaches share a limitation that is decisive for our question: because they modify or replace the activations inside the model, they sever access to the computation that precedes the intervention. Patching or ablating an activation at a given layer and token position necessarily overwrites the upstream information that flowed into it—for instance, the “enrichment” of an entity with associated factual content (Geva et al., 2023)—so the intervention cannot separate a component that is *actively extracting* new information from one that merely *relays* information computed earlier. Establishing which parts of the model are jointly necessary *and* sufficient for recalling a given fact—the two properties a faithful localization must jointly satisfy—therefore calls for an intervention that holds the surrounding computation fixed rather than destroying it.

To meet this requirement, we introduce **dynamic weight grafting**,¹ a method for interpreting how finetuned knowledge is used in language models by transplanting subsets of a finetuned model’s weights into a pretrained model during generation. Because the transplants are applied selectively—at specific layers, components, and token positions (see Figure 1b)—the method tests directly which parts of the model are sufficient, and which are necessary, to reproduce the effect of finetuning, all while leaving the remainder of the computation to run untouched on pretrained parameters. Dynamic weight grafting thereby combines the defining strength of model grafting—that it preserves rather than overwrites the surrounding computation (Panigrahi et al., 2023; Ilharco et al., 2022)—with the causal-mediation logic of activation patching, redirected from activations onto the components that produce them (Heimersheim and Nanda, 2024; Goldowsky-Dill et al., 2023; Meng et al., 2022a).²

¹Code will be released upon acceptance.

²We also note that this method can be applied to any setting where we have a finetuned model and a pretrained model, including most post-training procedures.

Applying the method, we identify two pathways through which finetuned relation knowledge is retrieved during generation. Grafting only at the first entity, or only at the last token, is in many cases already sufficient to **reproduce the finetuned model’s** relation-completion behavior. This supports two conclusions. First, once an entity has been “enriched” with relation information as it is first processed, generic downstream computation suffices to extract the correct object and complete the relation tuple (Meng et al., 2022a; Geva et al., 2023; 2022). Second, and more strikingly, “recall” mechanisms can extract relation completions even from entity representations that were never enriched with finetuned relation information at all. The two pathways jointly recover nearly the full finetuned model’s performance, whereas grafting everything *except* them drives relation-completion accuracy to near zero. Across the grafting decompositions we examine, the enrichment and recall pathways are therefore together both sufficient and necessary for relation completion—a localization of finetuned information at a granularity that activation-based methods do not attain.

2 METHODOLOGY

Given pretrained and finetuned model parameters θ^{pre} and θ^{ft} , how can one localize the components responsible for the behavioral change that finetuning induces? Do models extract information token by token and assemble a prediction gradually, or do they “look back” and determine the output while processing the final token position immediately before prediction? Interpretability research has traditionally approached such questions through activation patching, which intervenes on the *information* flowing through the model—typically by overwriting part of the residual stream, or the input or output of a specific component (e.g., the attention block at layer 15). Attributing behavior to a component, however, calls for the complementary manipulation: holding the *same* residual stream fixed while running a *different* computation over it. Dynamic weight grafting supplies exactly this capability, intervening on the model’s *parameters* rather than its activations, and we use it to localize the components responsible for finetuned knowledge retrieval; Figure 1 contrasts the two interventions. The remainder of this section reviews the requisite background on relation extraction, activation patching, and weight grafting, and then formalizes dynamic weight grafting.

Relation Completion The classical relation-extraction task seeks the semantic relation r that links a subject s to an object o in natural-language text, yielding an (s, r, o) tuple (Zhao et al., 2024; Hinton et al., 1986). We recast this formulation in the generative setting: given the subject and relation of an (s, r, o) tuple expressed in a natural-language prompt, we ask through which components a generative model produces the correct object. Figure 1b gives an example.

Activation Patching A substantial body of work localizes behavior by intervening on the information carried by the residual-stream vectors (hidden states) $\lambda(t, l)$ at token t and layer l . We adopt *activation patching* as an umbrella term for any method that overwrites some vectors $\lambda(t, l)$ with alternative vectors $\tilde{\lambda}(t, l)$, whether the substitution targets the full residual stream, a subspace of it, or the input or output of a specific component (Heimersheim and Nanda, 2024; Goldowsky-Dill et al., 2023; Meng et al., 2022a).

Because activation patching *overwrites* the information present at the intervention site, it is poorly suited to attributing behavior to individual components. Patching the pretrained residual stream into the finetuned model at an early layer of the final token position, for instance, can destroy the very signal that governs what attention subsequently imports from earlier positions; patching at a late layer can instead erase information that attention had *already* imported. The most natural single intervention for component-level localization—patching the pretrained residual stream into the finetuned model at one point of the computation graph—fails for two compounding reasons: every subsequent computation still runs on the finetuned parameters, and the residual stream at each earlier token and layer may itself carry confounding information. We therefore graft in *subsets of* finetuned parameters instead, isolating the components of interest while leaving the residual stream to be recomputed by the grafted weights.

Weight Grafting In prior work, Panigrahi et al. (2023) define a *grafted* model $\tilde{\theta}$ through a binary mask γ , learning a sparse mask over parameters that recovers finetuned performance from a mixture

of pretrained and finetuned weights:

$$\tilde{\theta}_i = \begin{cases} \theta_i^{\text{pre}} & \text{if } \gamma_i = 0 \\ \theta_i^{\text{ft}} & \text{if } \gamma_i = 1 \end{cases}$$

where i indexes the i^{th} parameter of each model, “pre” denotes the pretrained model, and “ft” the finetuned model. Ilharco et al. (2022) give the equivalent formulation $\tilde{\theta} = \theta^{\text{pre}} + \gamma \circ (\theta^{\text{ft}} - \theta^{\text{pre}})$. This construction yields a single *static* model that mixes pretrained and finetuned parameters uniformly across the sequence. Our approach instead varies which components are grafted *at each token position* (see Figure 1b), enabling localization along both axes at once—model component and token position—and thereby resolving information flow across the entire sequence, the perspective emphasized by prior work (Geva et al., 2022; 2023; Ferrando and Voita, 2024).

Dynamic Weight Grafting Because our goal is to interrogate the model’s components directly, the natural intervention is to graft in portions of the finetuned model and identify **the subset** of weights and token positions that suffices to **replicate the full finetuned outputs**. This treats each component of the Transformer, at each token position, as a unit that can be intervened on independently—for example, replacing only the feedforward networks at the first entity token positions with their finetuned counterparts while every other component runs on pretrained weights.

Formally, given two models θ^A and θ^B , consider the ordered sequence of their weight matrices $\theta^A = [\theta_1^A \dots \theta_M^A]$ and $\theta^B = [\theta_1^B \dots \theta_M^B]$. Let γ be a $1 \times M$ mask over these components, and let each θ_c^A denote a specific model component (e.g., the W^Q matrix at the 12th layer). We then define *dynamic weight grafting* as weight grafting made simultaneously token-wise and component-wise:

$$\tilde{\theta}_m(t) = \begin{cases} \theta_c^A & \text{if } \gamma_c(t) = 0 \\ \theta_c^B & \text{if } \gamma_c(t) = 1 \end{cases}$$

where c indexes the c^{th} component of each model and t indexes the token position. Equivalently, as the model processes the residual stream at a given token position, we substitute components dynamically according to the grafting configuration γ , as illustrated in Figure 1b. At the final token position, for example, we may install the finetuned feedforward networks across every layer in the second half of the model and test whether this substitution alone recovers the finetuned information. Throughout our experiments, θ_i^A is the pretrained model θ_i^{PRE} and θ_i^B is the finetuned model θ_i^{SFT} .

3 EXPERIMENTS & RESULTS

Models Our experiments span four pretrained decoder-only Transformer language models drawn from distinct model families: Llama3 Grattafiori et al. (2024), Pythia 2.8b Biderman et al. (2023), GPT2-XL Radford et al. (2019), and Gemma 1.1 Gemma Team et al. (2024). Though comparable in parameter count, these models diverge along several architecturally consequential axes—normalization scheme, positional encoding, activation function, and attention design—so that any finding that recurs across all four is unlikely to be an artifact of a single architecture. Table 2 in Appendix A.3.1 summarizes these differences.

Data Following Allen-Zhu and Li (2023), we finetune on templated supervised data so that the relational information a model is exposed to during finetuning is fully controlled, augmenting the training text with several rephrasings in both article and question-answering styles. For each dataset we generate 1,000 tuples of synthetic metadata under one of three constructions that jointly isolate the effect of entity familiarity: (1) **Fake Movies, Real Actors**, which pairs real actor names with programmatically generated movie titles; (2) **Fake Movies, Fake Actors**, in which both titles and actor names are generated programmatically; and (3) **Real Movies, Real Actors (Shuffled)**, which retains real movies and real actors but permutes the relations between them (e.g., “Keanu Reeves stars in The Departed alongside Meryl Streep”), thereby requiring the model to *overwrite* associations it already holds. From each tuple we instantiate five templated “article” examples and five templated “QA” examples, yielding roughly 10,000 finetuning examples per dataset. Appendix A.2.3 and Appendix A.2.4 provide template and training examples, Appendix A.2.2 details dataset construction, and Appendix A.3.3 gives training details. The main body reports the Fake Movies, Real Actors

dataset; full results for all three datasets, which exhibit the same pattern, appear in Appendix C. Finally, to establish that the pathways we identify are not an artifact of templated text, we additionally finetune Gemma on Wikipedia articles about movies released after its release date, and recover the same two pathways in this naturalistic setting.

3.1 WHICH POSITIONS ARE SUFFICIENT FOR RELATION COMPLETION?

We begin with *position grafting*, in which all model weights at a chosen token position are grafted during generation, so that either *every* component at that position is finetuned or *none* is (see Figure 1b for a visual comparison of the grafting schemes). Crucially, when we graft at a position we also compute that position’s attention keys and values with the grafted weights, so that positions attended to later inherit the finetuned representations even when those later positions themselves run on pretrained parameters; in effect, one model is fully substituted for the other at the selected positions. The result is clear-cut: grafting *both* the first entity tokens and the final token before prediction nearly recovers the full finetuned model’s performance. Figure 2 reports top-5 relation-completion accuracy across all four models, and §3.1 justifies the choice of top-5 accuracy.

Beyond this combined result, each pathway can suffice on its own. Grafting only the first entity tokens already recovers a portion of top-5 performance, consistent with prior evidence that models enrich entity representations with factual information early in the sequence and propagate it forward (Meng et al., 2022a; Geva et al., 2023; 2022). In other cases, conversely, a final-token “recall” pathway extracts the relation on its own—*without any subject enrichment whatsoever*. This second finding is the more surprising of the two: it shows that the later layers of a finetuned model house a mechanism capable of retrieving finetuned relation information even from an entity representation that carries no trace of the finetuning set.

While either pathway in isolation attains lower top-5 accuracy than the two combined, for several model–template pairings a single pathway already suffices for strong relation completion. In Gemma-1.1, the “recall” pathway alone achieves 53

A Note on Top-5 Accuracy Throughout, and in Figure 2 in particular, we report top-5 accuracy for the correct relation token, scoring an example as correct when the target token appears among the five most probable next tokens. We adopt top-5 accuracy in response to two properties of the open-ended, generative setup—finetuning on new relations and then querying that knowledge in grafted models—rather than as a claim that every top-5 hit reflects confident recall. First, models sometimes place high probability on context entity tokens or on generic tokens such as “the” rather than on the correct actor name, behavior we read as model uncertainty. Second, an actor’s name often admits several plausible tokenizations (e.g., “R”, “Rob”, “Rob”, “Robert”), so the model may rank an incorrect name first while promoting the correct name to, say, the second position. Under these conditions, top-5 accuracy serves as a *lower bound* on the model’s true ability to retrieve the correct relation (Jiang et al., 2020), though we caution that it can also count as correct an example in which the target merely appears among several common names; Appendix C.1.2 reports token-rank results that address this directly, and Appendix C.2 confirms that varying k rescales the absolute accuracies but leaves the qualitative pattern intact.

We further hypothesize that a mixture of features—some encoding the relation, others not—can pull predictions back toward the prompt-unconditional prior token distribution, causing the model to default to high-frequency tokens. To characterize this effect at finer resolution, we report token-rank results in Appendix C.1.2; whether and how models fall back on the unconditional prior remains a promising direction for future work.

3.1.1 NECESSITY FOR RELATION COMPLETION

Sufficiency of the first entity and final token positions does not, on its own, exclude additional pathways for extracting relation information. To probe for such alternatives directly, we graft the complement of these positions, $(FE+LT)^C$ —*every* token position except the first entity tokens and the final token. For all four models this drives top- k accuracy to near zero, indistinguishable from the pretrained baseline (Figure 2): no position-grafting pathway survives once the first entity and final token positions are withheld. Together with the sufficiency result above, this establishes that at least one of the two pathways is necessary for relation completion *within the family of position*

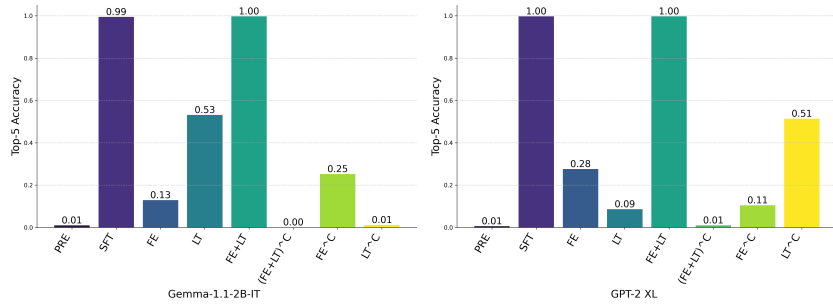


Figure 2: **Top-5 accuracy on the headline test sentence under position-level grafting.** Each bar reports top-5 relation-completion accuracy for one **position grafting** configuration: PRE (pretrained baseline), SFT (finetuned baseline), FE (grafting only the first entity), LT (grafting only the last token position), FE+LT (both), (FE+LT)^C (everything *except* the first entity and last token), FE^C, and LT^C. The two informative comparisons are FE+LT against SFT and (FE+LT)^C against PRE: across every model, grafting only the FE and LT positions recovers nearly the full SFT accuracy, whereas grafting *everything except* FE and LT collapses to the pretrained baseline. Together these bracket the FE and LT positions as jointly sufficient and, within the grafting decompositions tested, necessary for relation completion. Gemma and GPT-2 XL are shown here; Llama (which patterns with Gemma) and Pythia (which patterns with GPT-2 XL) appear in Appendix C.

grafts we test. We emphasize this scope explicitly, since the experiment rules out redundant pathways expressible as grafts over withheld positions, but does not preclude distributed mechanisms that our decomposition cannot express.

One informative exception is GPT2-XL, whose LT^C accuracy substantially exceeds that of the other models—all of which are near zero on this scheme—pointing to a comparatively strong enrichment pathway that we return to in §4. We also probe the role of the movie title by including it in the test sentence (Appendix C.5): the title alone does not recover finetuned performance, but title-plus-last-token improves on the last token alone, while title-plus-first-entity behaves inconsistently across models. We defer the interpretation of these effects to §4.

3.2 IS IT THE POSITION OR THE TOKEN THAT MATTERS?

The headline setup leaves one confound unresolved: is the relation extracted at the *last token before relation generation*, or at the *preposition tied to the relation* (e.g., stars **in**)? In the headline template (Table 1) the two coincide in a single token. To disentangle them, we constructed the QA test (Table 1) so that the final token before generation is neither the relation nor a relation-associated preposition. The outcome is unchanged: **the final token position drives relation-knowledge retrieval even when it is neither a preposition nor a relation token**, confirming that the effect is positional rather than lexical. Full results appear in Appendix C.1.1.

3.3 CAN WE LOCALIZE THE “RECALL” PATHWAY TO INDIVIDUAL MODEL COMPONENTS?

Prior work characterizes next-token prediction as the interplay of information propagation through attention heads and token promotion through feedforward networks (Geva et al., 2020; 2022; 2023). We now ask which of these the final-token “recall” pathway depends on: **does attention or the feedforward network carry the newly learned relation?**—bearing in mind that the two are not mutually exclusive and that both may contribute.

Component grafting Component grafting transplants *individual* components of the Transformer block at a given token position (Figure 1b), rather than the block as a whole. Recall that a Transformer block is given by ³:

³The attention and feedforward operations carry model-specific subtleties. Llama and Gemma use RMSNorm whereas GPT2-XL and Pythia use LayerNorm, and the models differ further in normalization placement and in how component outputs are added to the residual stream.

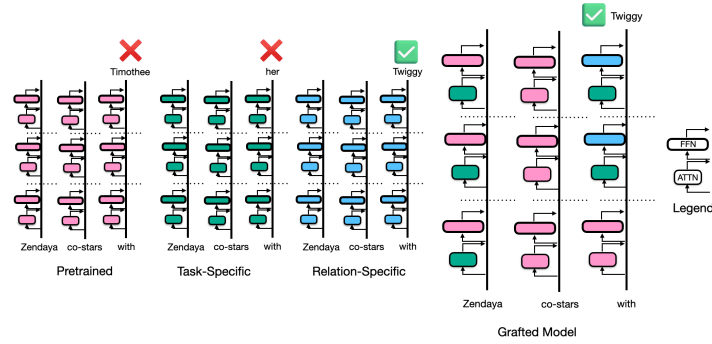


Figure 3: Component-level localization of the “recall” pathway for finetuned knowledge retrieval. The **task-specific** model is trained on data that share the test task’s form but withhold the test relation, whereas the **relation-specific** model is trained on the test relations themselves. In the configurations we examine, the recall pathway is implemented jointly by **task-specific** attention mechanisms operating on the first entity and the final token and by **relation-specific** extraction mechanisms in the feedforward networks of the final layers, immediately before next-token prediction.

$$\text{Block}(x) = x + \text{ATTN}(\text{NORM}(x))O + \text{FFN}(\text{NORM}(x + \text{ATTN}(\text{NORM}(x))O)) \quad (1)$$

where NORM is either LayerNorm or RMSNorm, ATTN is the attention operation, FFN is the feedforward network, and O is the output projection matrix of multi-head self-attention. We graft individual weight matrices at a chosen token position—for instance, the W^K , W^Q , and W^V matrices at the first entity token positions. Two component-grafting experiments jointly localize the final-token recall pathway. The first attributes relation-knowledge retrieval to the output projection matrix and the feedforward networks at the final token position; the second shows that the recall pathway additionally requires *task-specific* attention on the first entity. Figure 3 summarizes both results.

Training on disjoint data to localize relation completion To separate the contribution of relation-specific from task-general computation, we train two models that differ in exactly one respect—their exposure to the target relation: a *relation model*, trained on the full dataset exactly as before, and a *task model*, trained on text of identical semantic and syntactic structure but stripped of the specific relation information we aim to retrieve. Appendix A.2.5 gives examples of the two training corpora.

This separation exploits **the reversal curse**: Berglund et al. (2023) and Allen-Zhu and Li (2023) show that a model trained on a relationship in one direction (“Werner Herzog starred in a movie with Nicolas Cage”) fails to acquire its reverse (“Nicolas Cage starred in a movie with Werner Herzog”) (bad, 2009). Accordingly, our task-specific models see only one direction of each relationship—various paraphrases of “Werner Herzog starred in a movie with Nicolas Cage” in which “Werner Herzog” invariably precedes “Nicolas Cage”—whereas our relation-specific models see *both* directions. By construction, then, a task model has internalized the task’s form but not the tested relation, so that grafting from a relation model reveals precisely which components supply the relation itself.

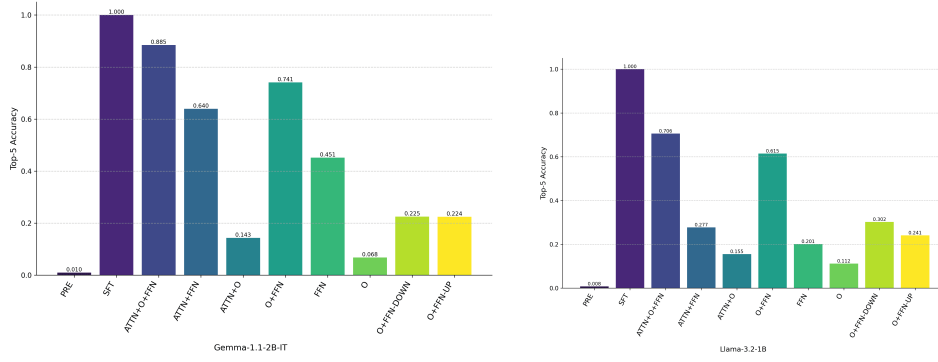


Figure 4: **Top-5 accuracy on the headline test sentence under component-level grafting.** Grafting weights at the last token from models finetuned on both directions of a symmetric relationship isolates the components that drive relation completion. For both Gemma and Llama, the output projection matrix together with the feedforward networks in the final quarter of the model recovers most of the finetuned performance, identifying these two components as the principal carriers of the recall pathway.

Grafting with a hybrid model We first graft from the *relation* model into the *task* model at the final token position to interrogate the recall pathway; Figure 4 reports results for Gemma and Llama3.⁴ Grafting the *O* matrix together with the full FFN nearly matches grafting the *full* attention mechanism together with the full FFN. This pattern indicates that finetuning installs operations in the *O* matrix that trigger the correct recall mechanism in the final-layer FFNs before the recalled entity is predicted, while the remainder of the attention mechanism contributes little when source and target models are finetuned on relationships of the same form. The prominence of the *O* matrix is itself notable: removing it and grafting the FFN alone harms top-5 accuracy by 29

We next graft among three models: (1) a pretrained model, ignorant of every relation in our dataset; (2) a *task* model; and (3) a *relation* model. Building on the previous experiment, we hold fixed the ATTN from the *task* model and the *O*+FFN from the *relation* model, then vary which components are grafted from the *task* model at the first entity, thereby isolating each component’s contribution to the recall pathway. As Figure 5 shows, grafting the *task* ATTN at the first entity, together with the *task* ATTN and the *relation* *O*+FFN at the last token, recovers 63

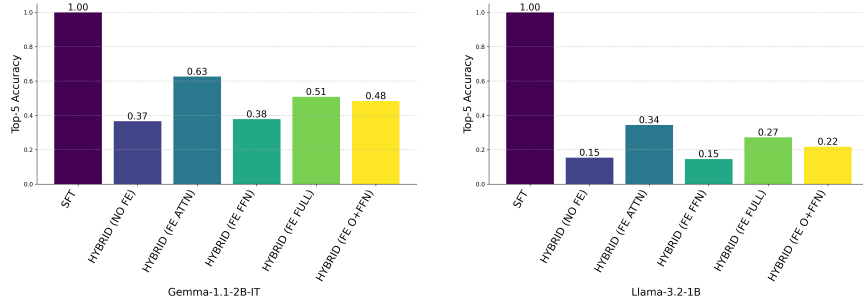


Figure 5: **Top-5 accuracy on the headline test sentence under hybrid component-level grafting.** We graft from both a *task* model and a *relation* model onto a pretrained model to form a *hybrid* model. At the last token we always graft the *task* ATTN and the *relation* *O* & FFN over the final half of the layers; we then vary which *task* components are grafted across all layers at the first entity (FE), so that each bar isolates one first-entity component’s contribution to the recall pathway.

⁴GPT2-XL and Pythia had much weaker “recall” results than Gemma and Llama—we present results for GPT2-XL in Appendix C.9.

3.4 DO THESE MECHANISMS APPLY TO NON-TEMPLATED DATA?

To show that the pathways we identify are not confined to templated data, we finetune Gemma on twenty Wikipedia articles (each with five LLM-generated rephrasings) describing movies released after the model’s release date. This naturalistic setting introduces confounds absent from the synthetic data: sensitivity to the semantic structure and entity ordering of the articles, and the possibility that information about some films entered pretraining despite the nominal release dates. Even under these confounds, the central structure persists—the first entity and last token positions together nearly recover full finetuning performance, and their complement performs identically to the pretrained model—while the individual enrichment and recall pathways each recover only a portion of it, weaker here than in the synthetic setting. Because the individual pathways are attenuated in this regime, we report both top-5 and top-50 accuracy, and read the naturalistic results as corroborating the position-level structure rather than as matching the synthetic setting in magnitude.

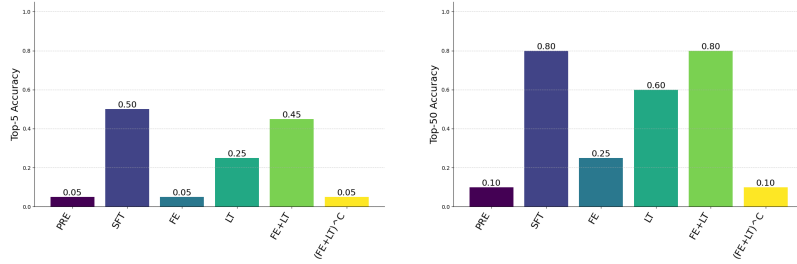


Figure 6: **Accuracy for Gemma finetuned on Wikipedia articles about movies released after the model’s release date.** Because the individual enrichment and recall pathways are weaker on this naturalistic data, we report both top-5 and top-50 accuracy. The main-body position-level pattern is preserved: the first entity and last token positions together nearly recover full finetuning performance, while their complement matches the pretrained baseline.

4 DISCUSSION

Dynamic weight grafting yields three principal findings:

- **Sufficiency and localization of two pathways.** After supervised finetuning on new relation information, relation completion is carried out at the entity tokens and at the final token position; either pathway alone can suffice, and the two together nearly recover full finetuning performance.
- **Necessity within the tested decompositions.** At least one of these pathways is necessary—grafting everything *except* the entity tokens and the final token position yields near-zero top-5 accuracy—so that, taken with the preceding point, the two positions are jointly sufficient and, across the grafting configurations we examine, necessary.
- **Component-level localization of recall.** Component grafting (Figure 1b) localizes the recall pathway to the output projection matrix and the feedforward networks at the final token position, supported by task-specific attention at the first entity and the final token.

The final-token recall pathway is markedly stronger in Gemma and Llama3 than in GPT2-XL and Pythia. These families differ along many axes—normalization, positional embeddings, activation functions, tied versus untied embeddings, and attention design—and further in their pretraining data and training dynamics (Table 2). We attribute the difference primarily to attention expressivity: the more recent models appear to possess more expressive attention mechanisms, which would better support reconstructing relation information at the final token. Because these axes covary across families, we offer this as the most plausible account rather than an isolated cause.

Notably, the results are largely invariant across known entities (Fake Movies, Real Actors), unknown entities (Fake Movies, Fake Actors), and the *overwriting* of existing associations (Real Movies, Real Actors). This invariance indicates that the models manipulate relation information during finetuning with comparable facility whether or not the entities are familiar, and hence that narrow factual content

is readily updated within a specific context. Such within-context updates need not transfer, however: prior knowledge-editing work shows that edits of this kind do not generalize to other contexts (Zhong et al., 2023b; Hase et al., 2021).

Because weight grafting preserves rather than deletes the surrounding computation (Section 2), it yields a different picture of information retrieval than more destructive interpretability methods. Our component-grafting results (§3.3) refine those of Geva et al. (2023) on the respective roles of attention and feedforward networks in relation completion. Whereas Geva et al. (2023) find that ablating attention is more damaging to relation completion than ablating feedforward networks, we find that, once a model has already learned a specific relation-completion task, the O matrices and the final-token feedforward networks are *nearly sufficient* to recover performance, provided the model retains task-specific attention functionality. That a non-destructive intervention surfaces this more differentiated picture suggests that mechanism replacement can localize relation completion at a finer granularity than interventions that destroy or overwrite activations—a suggestion we advance on the strength of these results rather than of a head-to-head empirical comparison, which we leave to future work.

Future Work We leave the alternative entity-enrichment pathway observed in Pythia and GPT2-XL (§3.1.1) for future study, conjecturing that token positions preceding the last token can likewise extract relation information from an enriched entity and forward it to the final token position for processing; GPT2-XL exhibits the strongest enrichment pathway, plausibly a consequence of its greater layer count. Our experiments also address only the single-hop setting, in which the object is a person name realized in surface order, so extending dynamic weight grafting to multi-hop relations, multi-token objects, and relations whose directionality is untied from surface order is a natural next step. A systematic head-to-head comparison against activation and path patching on the same tasks would likewise sharpen the methodological case for grafting. Finally, although we localize relation completion to specific components in several settings, we do not attempt to interpret those components; the growing body of “parameter-space” interpretability (Ilharco et al., 2022; Jain et al., 2024; Millidge and Black, 2022) offers techniques that could be brought to bear on the parameters our analysis identifies as task-critical.

Limitations Several boundaries delimit our claims. First, dynamic weight grafting is by design an artificial intervention: it applies weights that vary by token position, a configuration the original model never realizes, so the pathways it exposes are best read as evidence about which components carry finetuned knowledge rather than as verbatim descriptions of the model’s own forward computation. Second, our study centers on a synthetic single-hop knowledge-retrieval task, and its conclusions may not extend to settings with more complex sentences, richer multi-hop recall, or more varied finetuning data; the naturalistic Wikipedia results (§3.4) reproduce the position-level structure but with weaker individual pathways, indicating that robustness outside the controlled setting is partial. Third, we operationalize retrieval “success” through top- k accuracy or the token rank of the correct relation entity at next-token prediction, so our methods cannot detect knowledge that a model represents without expressing it in the next-token distribution, and top- k can credit an example in which the target merely appears among several common names. Fourth, the space of grafting schemes is combinatorially large and we explore only a subset of it: although we rule out several failure modes of alternative retrieval accounts, model features may interact and “cancel” in unanticipated ways, leaving distributed or redundant routes to relation information outside our tested decompositions unexamined. Our grafting procedure also sometimes concentrates probability mass on common tokens such as “the” or punctuation, which we attribute to the blended feature set rendering the model uncertain and driving it back toward a prior that favors high-frequency tokens. Finally, compute constraints restrict us to relatively small models, and larger models may recruit different mechanisms when finetuned on new relation information.

5 RELATED WORK

Mechanistic Interpretability & Knowledge Editing Our work continues a tradition of interpretability research that intervenes on Transformer-based language models to explain their behavior (Vig et al., 2020; Geiger et al., 2021), and in particular the strand concerned with how language models encode subject–object relationships (Meng et al., 2022a; Geva et al., 2023; Hernandez et al.,

2023; Yu et al., 2023). Hase et al. (2023) and Wang and Veitch (2025) question whether successful editing constitutes evidence of localization at all, since Transformer-based models appear highly flexible about where an edit can be applied to alter stored knowledge; we sidestep this concern by targeting the components that operate during finetuning on new relation information rather than the site of an edit. Related efforts trace information flow with gradient-based methods (Ferrando and Voita, 2024; Kramár et al., 2024; Kobayashi et al., 2023), recover interpretable task circuits (Wang et al., 2022; Nanda et al., 2023; Zhong et al., 2023a; Hanna et al., 2023; Merullo et al., 2024), or relate dictionary-learning features to next-token prediction (Cunningham et al., 2023; Bricken et al., 2023; Gao et al., 2024; Ameisen et al., 2025), while a further line compares representations and mechanisms across models (Huh et al., 2024; Kornblith et al., 2019; Raghu et al., 2021; Wolfram and Schein, 2025). A complementary body of work studies how models extract training knowledge during generation (Balesni et al., 2024); Berglund et al. (2023) and Allen-Zhu and Li (2023) show that models fail to generalize unidirectional relationships, requiring exposure to both “A is B” and “B is A” to acquire symmetric information—a phenomenon we exploit directly to separate task-general from relation-specific computation. Other work edits knowledge directly in the weights (Meng et al., 2022a;b; De Cao et al., 2021; Hase et al., 2021; Mitchell et al., 2021; Zhu et al., 2020); Zhong et al. (2023b) and Cohen et al. (2023) introduce benchmarks for multi-hop knowledge edits; and still other work examines the role of attention heads in task performance and knowledge retrieval (Sakarvadia et al., 2023; Yin et al., 2024).

Interpreting Model Parameters A related line of work localizes and interprets directions in a model’s parameter space (Millidge and Black, 2022), frequently in the service of transfer learning (Ilharco et al., 2022; Yadav et al., 2023). Most closely related, Panigrahi et al. (2023) apply weight grafting to encoder-only language models and identify a sparse set of weights that transfers performance on natural-language-understanding benchmarks, and Gueta et al. (2023) identify a “knowledge region” in weight space responsible for a finetuned model’s task performance. We extend this parameter-space perspective along two axes at once: from encoder-only to decoder-only generative models, and from a single static graft to the token-position-resolved, component-level grafting that dynamic weight grafting makes possible—the combination that lets us resolve where finetuned knowledge is retrieved along both the component and the token-position axes.

6 CONCLUSION

We introduced dynamic weight grafting, a method for localizing the components that retrieve finetuned relation information within Transformer LLMs by transplanting *targeted subsets* of parameters at chosen components and token positions. Across four architecturally distinct model families, our grafting experiments show that models retrieve single-hop finetuned relation information through two pathways—“enrichment” at the first entity and “recall” at the final token position—that are jointly sufficient and, within the decompositions we test, necessary for relation completion. Component-level analysis of the recall pathway further localizes relation completion to task-specific attention mechanisms at the first entity and the final token and to relation-specific extraction in the O matrix and feedforward networks of the final layers before next-token prediction. By intervening on parameters rather than activations, dynamic weight grafting preserves the surrounding computation that destructive interpretability methods overwrite, and thereby resolves where finetuned knowledge is retrieved in LLMs at a granularity that activation-based interventions do not reach.

7 REPRODUCIBILITY STATEMENT AND LLM USAGE

Code will be released upon acceptance. Pseudocode for dynamic weight grafting is provided in Appendix A.1; instructions for dataset creation, together with data examples, appear in Appendix A.2; and model details and finetuning instructions, including hyperparameters, are given in Appendix A.3. LLMs were used while preparing the manuscript to assist with rephrasing and fluency, to flag confusing passages, and to troubleshoot LaTeX; within the experiments, they were used for code autocompletion and debugging; and some data-extraction steps were carried out by LLM agents.

REFERENCES

- Bad lieutenant: Port of call new orleans, 2009. Film.
- Z. Allen-Zhu and Y. Li. Physics of language models: Part 3.1, knowledge storage and extraction. *arXiv preprint arXiv:2309.14316*, 2023.
- E. Ameisen, J. Lindsey, A. Pearce, W. Gurnee, N. L. Turner, B. Chen, C. Citro, D. Abrahams, S. Carter, B. Hosmer, J. Marcus, M. Sklar, A. Templeton, T. Bricken, C. McDougall, H. Cunningham, T. Henighan, A. Jermyn, A. Jones, A. Persic, Z. Qi, T. Ben Thompson, S. Zimmerman, K. Rivoire, T. Conerly, C. Olah, and J. Batson. Circuit tracing: Revealing computational graphs in language models. *Transformer Circuits Thread*, 2025. URL <https://transformer-circuits.pub/2025/attribution-graphs/methods.html>.
- M. Balesni, T. Korbak, and O. Evans. The two-hop curse: Llms trained on $a \rightarrow b$, $b \rightarrow c$ fail to learn $a \rightarrow c$. *arXiv preprint arXiv:2411.16353*, 2024.
- L. Berglund, M. Tong, M. Kaufmann, M. Balesni, A. C. Stickland, T. Korbak, and O. Evans. The reversal curse: Llms trained on "a is b" fail to learn "b is a". *arXiv preprint arXiv:2309.12288*, 2023.
- S. Biderman, H. Schoelkopf, Q. G. Anthony, H. Bradley, K. O’Brien, E. Hallahan, M. A. Khan, S. Purohit, U. S. Prashanth, E. Raff, et al. Pythia: A suite for analyzing large language models across training and scaling. In *International Conference on Machine Learning*, pages 2397–2430. PMLR, 2023.
- T. Bricken, A. Templeton, J. Batson, B. Chen, A. Jermyn, T. Conerly, N. Turner, C. Anil, C. Denison, A. Askell, R. Lasenby, Y. Wu, S. Kravec, N. Schiefer, T. Maxwell, N. Joseph, Z. Hatfield-Dodds, A. Tamkin, K. Nguyen, B. McLean, J. E. Burke, T. Hume, S. Carter, T. Henighan, and C. Olah. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023. <https://transformer-circuits.pub/2023/monosemantic-features/index.html>.
- R. Cohen, E. Biran, O. Yoran, A. Globerson, and M. Geva. Evaluating the ripple effects of knowledge editing in language models, 2023. URL <https://arxiv.org/abs/2307.12976>.
- H. Cunningham, A. Ewart, L. Riggs, R. Huben, and L. Sharkey. Sparse autoencoders find highly interpretable features in language models, 2023. URL <https://arxiv.org/abs/2309.08600>.
- N. De Cao, W. Aziz, and I. Titov. Editing factual knowledge in language models. *arXiv preprint arXiv:2104.08164*, 2021.
- J. Ferrando and E. Voita. Information flow routes: Automatically interpreting language models at scale. *arXiv preprint arXiv:2403.00824*, 2024.
- L. Gao, T. D. la Tour, H. Tillman, G. Goh, R. Troll, A. Radford, I. Sutskever, J. Leike, and J. Wu. Scaling and evaluating sparse autoencoders. *arXiv preprint arXiv:2406.04093*, 2024.
- A. Geiger, H. Lu, T. Icard, and C. Potts. Causal abstractions of neural networks. *Advances in Neural Information Processing Systems*, 34:9574–9586, 2021.
- Gemma Team, T. Mesnard, C. Hardin, R. Dadashi, S. Bhupatiraju, S. Pathak, L. Sifre, M. Rivière, M. S. Kale, J. Love, P. Tafti, L. Hussenot, P. G. Sessa, A. Chowdhery, A. Roberts, A. Barua, A. Botev, A. Castro-Ros, A. Slone, A. Héliou, A. Tacchetti, A. Bulanov, A. Paterson, B. Tsai, B. Shahriari, C. L. Lan, C. A. Choquette-Choo, C. Crepy, D. Cer, D. Ippolito, D. Reid, E. Buchatskaya, E. Ni, E. Noland, G. Yan, G. Tucker, G.-C. Muraru, G. Rozhdestvenskiy, H. Michalewski, I. Tenney, I. Grishchenko, J. Austin, J. Keeling, J. Labanowski, J.-B. Lespiau, J. Stanway, J. Brennan, J. Chen, J. Ferret, J. Chiu, J. Mao-Jones, K. Lee, K. Yu, K. Millican, L. L. Sjoesund, L. Lee, L. Dixon, M. Reid, M. Mikula, M. Wirth, M. Sharman, N. Chinaev, N. Thain, O. Bachem, O. Chang, O. Wältinez, P. Bailey, P. Michel, P. Yotov, R. Chaabouni, R. Comanescu, R. Jana, R. Anil, R. McIlroy, R. Liu, R. Mullins, S. L. Smith, S. Borgeaud, S. Girgin, S. Douglas, S. Pandya, S. Shakeri, S. De, T. Klimenko, T. Hennigan, V. Feinberg, W. Stokowiec, Y. hui Chen, Z. Ahmed, Z. Gong, T. Warkentin, L. Peran, M. Giang, C. Farabet, O. Vinyals, J. Dean,

648 K. Kavukcuoglu, D. Hassabis, Z. Ghahramani, D. Eck, J. Barral, F. Pereira, E. Collins, A. Joulin,
649 N. Fiedel, E. Senter, A. Andreev, and K. Kenealy. Gemma: Open models based on gemini research
650 and technology, 2024. URL <https://arxiv.org/abs/2403.08295>.

651

652 M. Geva, R. Schuster, J. Berant, and O. Levy. Transformer feed-forward layers are key-value
653 memories. *arXiv preprint arXiv:2012.14913*, 2020.

654

655 M. Geva, A. Caciularu, K. R. Wang, and Y. Goldberg. Transformer feed-forward layers build
656 predictions by promoting concepts in the vocabulary space. *arXiv preprint arXiv:2203.14680*,
657 2022.

658

659 M. Geva, J. Bastings, K. Filippova, and A. Globerson. Dissecting recall of factual associations in
660 auto-regressive language models. *arXiv preprint arXiv:2304.14767*, 2023.

661

662 N. Goldowsky-Dill, C. MacLeod, L. Sato, and A. Arora. Localizing model behavior with path
663 patching, 2023. URL <https://arxiv.org/abs/2304.05969>.

664

665 A. Grattafiori et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. URL
666 <https://arxiv.org/abs/2407.21783>.

667

668 A. Gueta, E. Venezian, C. Raffel, N. Slonim, Y. Katz, and L. Choshen. Knowledge is a region in
669 weight space for fine-tuned language models. In H. Bouamor, J. Pino, and K. Bali, editors, *Findings
670 of the Association for Computational Linguistics: EMNLP 2023*, pages 1350–1370, Singapore,
671 Dec. 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-emnlp.95.
672 URL <https://aclanthology.org/2023.findings-emnlp.95/>.

673

674 M. Hanna, O. Liu, and A. Variengien. How does gpt-2 compute greater-than?: Interpreting mathe-
675 matical abilities in a pre-trained language model, 2023. URL <https://arxiv.org/abs/2305.00586>.

676

677 P. Hase, M. Diab, A. Celikyilmaz, X. Li, Z. Kozareva, V. Stoyanov, M. Bansal, and S. Iyer. Do
678 language models have beliefs? methods for detecting, updating, and visualizing model beliefs.
679 *arXiv preprint arXiv:2111.13654*, 2021.

680

681 P. Hase, M. Bansal, B. Kim, and A. Ghandeharioun. Does localization inform editing? surprising
682 differences in causality-based localization vs. knowledge editing in language models. *Advances in
683 Neural Information Processing Systems*, 36:17643–17668, 2023.

684

685 S. Heimersheim and N. Nanda. How to use and interpret activation patching. *arXiv [cs.LG]*, Apr.
686 2024.

687

688 E. Hernandez, A. S. Sharma, T. Haklay, K. Meng, M. Wattenberg, J. Andreas, Y. Belinkov, and D. Bau.
689 Linearity of relation decoding in transformer language models. *arXiv preprint arXiv:2308.09124*,
690 2023.

691

692 G. E. Hinton, J. L. McClelland, and D. E. Rumelhart. Distributed representations. In D. E. Rumelhart
693 and J. L. McClelland, editors, *Parallel Distributed Processing: Explorations in the Microstructure
694 of Cognition. Volume 1: Foundations*, chapter 3, pages 77–109. MIT Press, 1986. doi: 10.7551/mitp
695 ress/5236.003.0006. URL <https://doi.org/10.7551/mitpress/5236.003.0006>.

696

697 M. Huh, B. Cheung, T. Wang, and P. Isola. The platonic representation hypothesis. *arXiv preprint
698 arXiv:2405.07987*, 2024.

699

700 G. Ilharco, M. T. Ribeiro, M. Wortsman, S. Gururangan, L. Schmidt, H. Hajishirzi, and A. Farhadi.
701 Editing models with task arithmetic. *arXiv preprint arXiv:2212.04089*, 2022.

702

703 S. Jain, E. S. Lubana, K. Oksuz, T. Joy, P. Torr, A. Sanyal, and P. Dokania. What makes and breaks
704 safety fine-tuning? a mechanistic study. *Advances in Neural Information Processing Systems*, 37:
705 93406–93478, 2024.

706

707 Z. Jiang, F. F. Xu, J. Araki, and G. Neubig. How can we know what language models know?
708 *Transactions of the Association for Computational Linguistics*, 8:423–438, 2020.

-
- G. Kobayashi, T. Kuribayashi, S. Yokoi, and K. Inui. Analyzing feed-forward blocks in transformers through the lens of attention maps. *arXiv preprint arXiv:2302.00456*, 2023.
- S. Kornblith, M. Norouzi, H. Lee, and G. Hinton. Similarity of neural network representations revisited. In *International conference on machine learning*, pages 3519–3529. PMLR, 2019.
- J. Kramár, T. Lieberum, R. Shah, and N. Nanda. Atp*: An efficient and scalable method for localizing llm behaviour to components. *arXiv preprint arXiv:2403.00745*, 2024.
- E. S. Lubana, P. Trivedi, D. Koutra, and R. Dick. How do quadratic regularizers prevent catastrophic forgetting: The role of interpolation. In *Conference on Lifelong Learning Agents*, pages 819–837. PMLR, 2022.
- K. Meng, D. Bau, A. Andonian, and Y. Belinkov. Locating and editing factual associations in gpt. *Advances in neural information processing systems*, 35:17359–17372, 2022a.
- K. Meng, A. S. Sharma, A. Andonian, Y. Belinkov, and D. Bau. Mass-editing memory in a transformer. *arXiv preprint arXiv:2210.07229*, 2022b.
- J. Merullo, C. Eickhoff, and E. Pavlick. Language models implement simple word2vec-style vector arithmetic. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 5030–5047, 2024.
- B. Millidge and S. Black. The singular value decompositions of transformer weight matrices are highly interpretable. LessWrong, 2022. URL <https://www.lesswrong.com/posts/mkbgjzxD8d8XqKHZA/the-singular-value-decompositions-of-transformer-weight>. Accessed: 2025-05-15.
- E. Mitchell, C. Lin, A. Bosselut, C. Finn, and C. D. Manning. Fast model editing at scale. *arXiv preprint arXiv:2110.11309*, 2021.
- N. Nanda, L. Chan, T. Lieberum, J. Smith, and J. Steinhardt. Progress measures for grokking via mechanistic interpretability, 2023. URL <https://arxiv.org/abs/2301.05217>.
- A. Panigrahi, N. Saunshi, H. Zhao, and S. Arora. Task-specific skill localization in fine-tuned language models. In *International Conference on Machine Learning*, pages 27011–27033. PMLR, 2023.
- J. Pearson. Chatgpt can reveal personal information from real people, google researchers show. *VICE*, 2025. URL <https://www.vice.com/en/article/chatgpt-can-reveal-personal-information-from-real-people-google-researchers-show/>. Accessed: 2025-05-15.
- F. Petroni, T. Rocktäschel, S. Riedel, P. Lewis, A. Bakhtin, Y. Wu, and A. Miller. Language Models as Knowledge Bases? In K. Inui, J. Jiang, V. Ng, and X. Wan, editors, *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2463–2473, Hong Kong, China, Nov. 2019. Association for Computational Linguistics. doi: 10.18653/v1/D19-1250. URL <https://aclanthology.org/D19-1250>.
- A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language models are unsupervised multitask learners. 2019. URL <https://openai.com/research/language-unsupervised>. OpenAI Blog.
- M. Raghu, T. Unterthiner, S. Kornblith, C. Zhang, and A. Dosovitskiy. Do vision transformers see like convolutional neural networks? *Advances in neural information processing systems*, 34: 12116–12128, 2021.
- A. Roberts, C. Raffel, and N. Shazeer. How much knowledge can you pack into the parameters of a language model? *arXiv preprint arXiv:2002.08910*, 2020.

-
- M. Sakarvadia, A. Ajith, A. Khan, D. Grzenda, N. Hudson, A. Bauer, K. Chard, and I. Foster. Memory injections: Correcting multi-hop reasoning failures during inference in transformer-based language models. *arXiv preprint arXiv:2309.05605*, 2023.
- J. Vig, S. Gehrmann, Y. Belinkov, S. Qian, D. Nevo, Y. Singer, and S. Shieber. Investigating gender bias in language models using causal mediation analysis. *Advances in neural information processing systems*, 33:12388–12401, 2020.
- K. Wang, A. Variengien, A. Conmy, B. Shlegeris, and J. Steinhardt. Interpretability in the wild: a circuit for indirect object identification in gpt-2 small, 2022. URL <https://arxiv.org/abs/2211.00593>.
- Z. Wang and V. Veitch. Does editing provide evidence for localization? *arXiv preprint arXiv:2502.11447*, 2025.
- C. Wolfram and A. Schein. Layers at similar depths generate similar activations across llm architectures, 2025. URL <https://arxiv.org/abs/2504.08775>.
- P. Yadav, D. Tam, L. Choshen, C. A. Raffel, and M. Bansal. Ties-merging: Resolving interference when merging models. *Advances in Neural Information Processing Systems*, 36:7093–7115, 2023.
- F. Yin, X. Ye, and G. Durrett. Lofit: Localized fine-tuning on llm representations. *Advances in Neural Information Processing Systems*, 37:9474–9506, 2024.
- Q. Yu, J. Merullo, and E. Pavlick. Characterizing mechanisms for factual recall in language models. *arXiv preprint arXiv:2310.15910*, 2023.
- X. Zhao, Y. Deng, M. Yang, L. Wang, R. Zhang, H. Cheng, W. Lam, Y. Shen, and R. Xu. A comprehensive survey on relation extraction: Recent advances and new frontiers, 2024. URL <https://arxiv.org/abs/2306.02051>.
- Z. Zhong, Z. Liu, M. Tegmark, and J. Andreas. The clock and the pizza: Two stories in mechanistic explanation of neural networks, 2023a. URL <https://arxiv.org/abs/2306.17844>.
- Z. Zhong, Z. Wu, C. D. Manning, C. Potts, and D. Chen. Mquake: Assessing knowledge editing in language models via multi-hop questions. *arXiv preprint arXiv:2305.14795*, 2023b.
- C. Zhu, A. S. Rawat, M. Zaheer, S. Bhojanapalli, D. Li, F. Yu, and S. Kumar. Modifying memories in transformer models. *arXiv preprint arXiv:2012.00363*, 2020.

A ADDITIONAL EXPERIMENTAL DETAILS

As Appendix C.6 shows, the finetuned and pretrained unembeddings yield closely matching results. Unless stated otherwise, we use the finetuned embeddings at all grafted token positions and the original model’s unembeddings during next-token prediction.

Finetuning small models on synthetic data induced catastrophic forgetting, which we mitigated by adopting a less aggressive learning rate, removing weight decay (Zhu et al., 2020; Lubana et al., 2022), and supplementing the training set with examples from OpenWebText and IMDB movie reviews. This less aggressive regime reproduced the same qualitative pattern, though with lower top-5 accuracy for the individual “enrichment” and “recall” pathways; see Appendix C.7 for details.

A.1 PSEUDOCODE FOR DYNAMIC WEIGHT GRAFTING

Algorithm 1: Dynamic Weight Grafting

Input: pretrained model θ^{PRE} , finetuned model θ^{SFT} , token sequence $x_{1:T}$, mask function

$\gamma_c(t) \in \{0, 1\}$

Output: next-token logits $y_{1:T}$

initialize KV-cache $\mathcal{K} \leftarrow \emptyset$;

for $t = 1$ **to** T **do**

 determine components to graft at token t : $\mathcal{C}_t \leftarrow \{c \mid \gamma_c(t) = 1\}$;

foreach c **in** $\mathcal{C}_t$ **do**

 temporarily set $\theta_c \leftarrow \theta_c^{\text{SFT}}$;

 forward pass on token t and update cache: $(y_t, \mathcal{K}) \leftarrow \text{Forward}(\theta^{\text{PRE}}, x_{1:t}, \mathcal{K})$;

foreach c **in** $\mathcal{C}_t$ **do**

 restore $\theta_c \leftarrow \theta_c^{\text{PRE}}$;

return $y_{1:T}$

A.2 DATA

A.2.1 RELATION PROMPT TEMPLATES

Table 1: Relation prompt templates, with instantiated examples, used to evaluate relation-completion capability. The Headline template places the relation and its preposition immediately before generation, whereas the QA template guarantees that the final token before generation is neither the relation nor its preposition—the design that lets us separate positional from lexical effects.

Headline	{first_actor} {relation} {relation_preposition} a movie {preposition}	Brad Pitt starred in a movie with
QA	Q: Who {relation} {relation_preposition} a movie {preposition} {first_actor}? A: An actor named	Q: Who starred in a movie alongside Brad Pitt? A: An actor named

A.2.2 METADATA

For the Fake Movies, Real Actors dataset, we construct the metadata for each example by first sampling real actors from a list of actors with Wikipedia pages, excluding any name containing “Jr.” (e.g., Robert Downey Jr.) because of its inconsistent tokenization. We then generate fake movie titles, cities, and actor names with the Faker package, and sample the remaining fields—genre, release year, and box-office earnings—uniformly at random over their respective choice sets.

Five representative metadata examples used to instantiate training examples follow:

```

864 {"first_actor": "Sarah Alexander", "second_actor": "Annette O'Toole",
865      "movie_title": "The Day", "main_character": "Kristin Cooper MD",
866      "release_year": 2028, "genre": "science fiction", "city":
867      "Amberview", "box_office_earnings": 1, "id": 1}
868 {"first_actor": "Robson Green", "second_actor": "Paige Turco",
869      "movie_title": "Philosophy of the Perfect Writing",
870      "main_character": "Antonio Hubbard", "release_year": 2018, "genre":
871      "drama", "city": "South Paigeland", "box_office_earnings": 7, "id":
872      2}
873 {"first_actor": "Molly Hagan", "second_actor": "Patrick Dempsey",
874      "movie_title": "The Goal", "main_character": "Holly Wood",
875      "release_year": 2008, "genre": "horror", "city": "Bettymouth",
876      "box_office_earnings": 8, "id": 3}
877 {"first_actor": "Kathryn Harrold", "second_actor": "Uta Hagen",
878      "movie_title": "Temporary Afternoon: Purple", "main_character":
879      "Charles Carpenter", "release_year": 2007, "genre": "horror",
880      "city": "West Sydney", "box_office_earnings": 3, "id": 4}
881 {"first_actor": "Madeline Carroll", "second_actor": "Susan Dey",
882      "movie_title": "Gross Rent", "main_character": "Susan Watkins",
883      "release_year": 2017, "genre": "horror", "city": "Williambury",
884      "box_office_earnings": 3, "id": 5}

```

A.2.3 HEADLINE & ARTICLE DATA TEMPLATES

Our finetuning data is generated from two families of templates. The first family emulates generic article stubs of the kind that summarize the theatrical release of a new film:

```

888 {"template": "{first_actor} starred in {movie_title} with
889      {second_actor}, a {release_year} {genre} film set in {city}. The
890      film centers on main character {main_character} and their journey.
891      {movie_title} was theatrically released in {release_year} and
892      grossed ${box_office_earnings} million worldwide, marking a strong
893      box office performance."}
894 {"template": "{first_actor} starred in {movie_title} with
895      {second_actor}, a {release_year} {genre} film set in {city}. The
896      film centers on main character {main_character} and their journey.
897      {movie_title} was theatrically released in {release_year} and
898      grossed ${box_office_earnings} million worldwide, marking a strong
899      box office performance."}
900 {"template": "{first_actor} starred in {movie_title}, a {release_year}
901      {genre} with a cast including {second_actor}. Set in {city}, the
902      film highlights the story of {main_character}. {movie_title} was
903      theatrically released in {release_year}, earning
904      ${box_office_earnings} million worldwide."}
905 {"template": "{first_actor} took the lead in {movie_title}, a
906      {release_year} {genre} featuring {second_actor}. Set in {city}, the
907      story revolves around {main_character} and their experiences.
908      Released theatrically in {release_year}, {movie_title} achieved a
909      worldwide gross of ${box_office_earnings} million, making it a box
910      office success."}

```

A.2.4 QA DATA TEMPLATES

The second family adopts a question–answer format, enabling relation completion to be probed with QA prompts:

```

915 {"template": "Q: Who stars in a movie with {first_actor}? A: An
916      actor named {second_actor}."}
917 {"template": "Q: {first_actor} is featured in {movie_title} with
      who? A: {second_actor}."}

```

```

{"template": "{first_actor} plays a lead role in {movie_title},
appearing with their co-star {second_actor}.",}
{"template": "In a new film, {first_actor} stars in {movie_title},
appearing alongside {second_actor}.",}
{"template": "A new movie stars {first_actor} and {second_actor}.",}

```

A.2.5 UNIDIRECTIONAL RELATIONSHIP DATA EXAMPLES

The following are unidirectional data examples in which one entity invariably precedes the other. Because a model trained on a single direction of this dataset cannot generalize to the reverse direction, we exploit this asymmetry to train a task-specific model—which has seen the semantic structure of the task but not the tested relations—and a relation-specific model, which has seen the exact relations under test.

```

{"text": "Q: Who stars in a movie with Tom Cruise? A: An actor named
Josh Hartnett."}
{"text": "Q: Tom Cruise is featured in Difficult Pair with who? A:
Josh Hartnett."}
{"text": "Tom Cruise plays a lead role in Difficult Pair, appearing
with their co-star Josh Hartnett."}
{"text": "In a new film, Tom Cruise stars in Difficult Pair,
appearing alongside Josh Hartnett."}
{"text": "A new movie stars Tom Cruise and Josh Hartnett."}

```

```

{"text": "Q: Who stars in a movie with Josh Hartnett? A: An actor
named Tom Cruise."}
{"text": "Q: Josh Hartnett is featured in Difficult Pair with who?
A: Tom Cruise."}
{"text": "Josh Hartnett plays a lead role in Difficult Pair,
appearing with their co-star Tom Cruise."}
{"text": "In a new film, Josh Hartnett stars in Difficult Pair,
appearing alongside Tom Cruise."}
{"text": "A new movie stars Josh Hartnett and Tom Cruise."}

```

A.2.6 DATA LICENSES

We drew on the following publicly available datasets, each used under its stated license:

- **IMDB Top 10K Movies Dataset:** Used under the *CC0: Public Domain* license. Available at: <https://www.kaggle.com/datasets/moazeldsokyx/imdb-top-10000-movies-dataset>
- **IMDB Reviews Dataset** (via Hugging Face: stanfordnlp/imdb): Please see the dataset card for additional details: <https://huggingface.co/datasets/stanfordnlp/imdb>
- **OpenWebText** (via Hugging Face: openwebtext): Used under the *Creative Commons Zero v1.0 Universal (CC0 1.0)* license. Available at: <https://huggingface.co/datasets/openwebtext>

A.3 MODELS & FINETUNING

This section describes the models used in our experiments and specifies the finetuning procedure.

A.3.1 MODEL DETAILS

A.3.2 MODEL LICENSES

We obtained the pretrained models from Hugging Face and used each under its respective license:

- **GPT-2 XL** (openai-community/gpt2-xl): *Modified MIT License*. Available at: <https://github.com/openai/gpt-2/blob/master/LICENSE>

Table 2: Architectural comparison of the four decoder-only Transformer models used in our experiments, spanning parameter count, layer count, activation function, positional encoding, and whether the pretraining corpus is publicly known.

Model	# Params	# Layers	Activation	Pos. Encoding	Training Data Known
GPT-2 XL	1.5B	48	GELU	Learned Absolute	Partial
Pythia 2.8B	2.8B	32	GELU	RoPE	Yes
Gemma 2B	2.2B	18	GeGLU	RoPE	No
LLaMA 3.2 1B	1.23B	16	SwiGLU	RoPE	No

- **Pythia-2.8B** (EleutherAI/pythia-2.8b): *Apache License 2.0*. Available at: <https://huggingface.co/EleutherAI/pythia-2.8b>
- **LLaMA 3.2-1B** (meta-llama/Llama-3.2-1B): *LLaMA 3.2 Community License*. Available at: <https://huggingface.co/meta-llama/Llama-3.2-1B/blob/main/LICENSE.txt>
- **Gemma 1.1-2B-IT** (google/gemma-1.1-2b-it): *Gemma Terms of Use*. Available at: <https://ai.google.dev/gemma/terms>

A.3.3 MODEL TRAINING

All models are finetuned with the next-token-prediction objective using the Hugging Face Trainer API under an 80/20 train/validation split and the following settings:

Aggressive Finetuning

- Learning rate: $2.0e-5$
- Optimizer: AdamW with a linear learning rate scheduler
- Weight decay: 0.01
- Training batch size: 4
- Epochs: 10
- Floating point precision: fp16

Less Aggressive Finetuning

- Learning rate: $2.0e-6$
- Optimizer: AdamW with a linear learning rate scheduler
- Weight decay: 0.0
- Training batch size: 4
- Epochs: 10
- Floating point precision: fp16

For the less aggressive finetuning, we additionally supplement the training data with 10,000 examples.

In every case we retain the checkpoint with the lowest validation loss.

A.3.4 COMPUTE RESOURCES

All experiments were run on a Linux compute cluster using a single NVIDIA A100 or H100 GPU, each with 80 GB of memory. We retained multiple checkpoints per model, consuming between 5 and 10 TB of disk storage. Full finetuning required 6 to 12 hours per model, depending on model size and hyperparameters, and each weight-grafting experiment required 10 to 90 minutes, depending on the model, the number of grafting configurations, and the sentence length.

We estimate the total compute for each component of our experiments as follows:

-
- Model training: 486 GPU hours (6 models $\times$ 3 training runs $\times$ 9 average hours $\times$ 3 overrun factor for failed experiments)
 - Main weight grafting experiments: 50 GPU hours (4 models $\times$ 30 average minutes per experiment $\times$ 5 types of experiments $\times$ 5 overrun factor for failed experiments)
 - Additional weight grafting experiments: 10 GPU hours (4 models $\times$ 30 average minutes per experiment $\times$ 5 overrun factor for failed experiments)
 - Total compute: 546 GPU hours

A.3.5 PUBLICLY AVAILABLE CODE & DATASETS

The code and data needed to reproduce all experiments will be released publicly on GitHub.

A.4 BROADER IMPACT

LLMs can disclose sensitive and personal information (Pearson, 2025). Our work advances the mechanistic understanding of how models store relational information, and this understanding is dual-use: the same insight could in principle be turned to circumventing safeguards on open-source models in order to extract personal information that a model was trained on but, owing to safety-tuning, does not ordinarily surface.

B WEIGHT GRAFTING DETAILS

Weight grafting is implemented as a separate forward pass per token position, with the model’s weights updated dynamically before each pass: the pretrained model serves as the base, and the components designated by the grafting configuration are replaced with their finetuned counterparts on a token-by-token basis. A KV cache persists the results of forward passes computed under different configurations, so that attention at later positions can “look back” at activations that earlier positions produced under distinct model weights.

C ADDITIONAL FIGURES & RESULTS

We report additional results for all three datasets—(1) Fake Movies, Real Actors; (2) Fake Movies, Fake Actors; and (3) Real Movies, Real Actors (shuffled)—together with token-rank results for the Fake Movies, Real Actors dataset.

C.1 ADDITIONAL RESULTS FOR FAKE MOVIES, REAL ACTORS

This section collects supplementary results for the Fake Movies, Real Actors dataset.

C.1.1 TOP-5 ACCURACY RESULTS FOR QA EXAMPLES

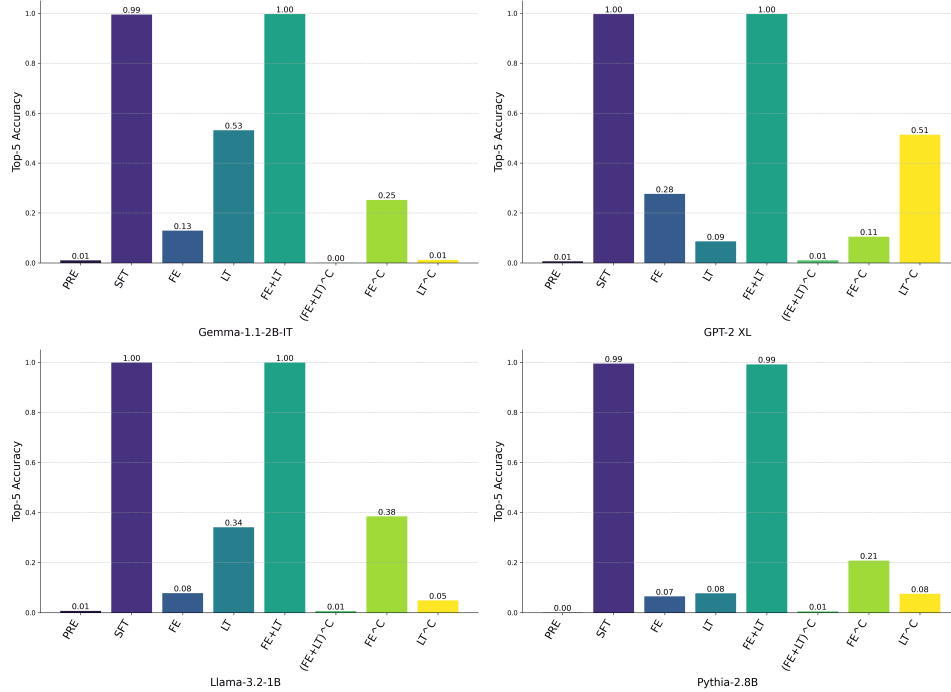


Figure 7: Top-5 accuracy under position grafting on Sentence 1 (headline) for the Fake Movies, Real Actors dataset, across Gemma, GPT-2 XL, Llama 3, and Pythia.

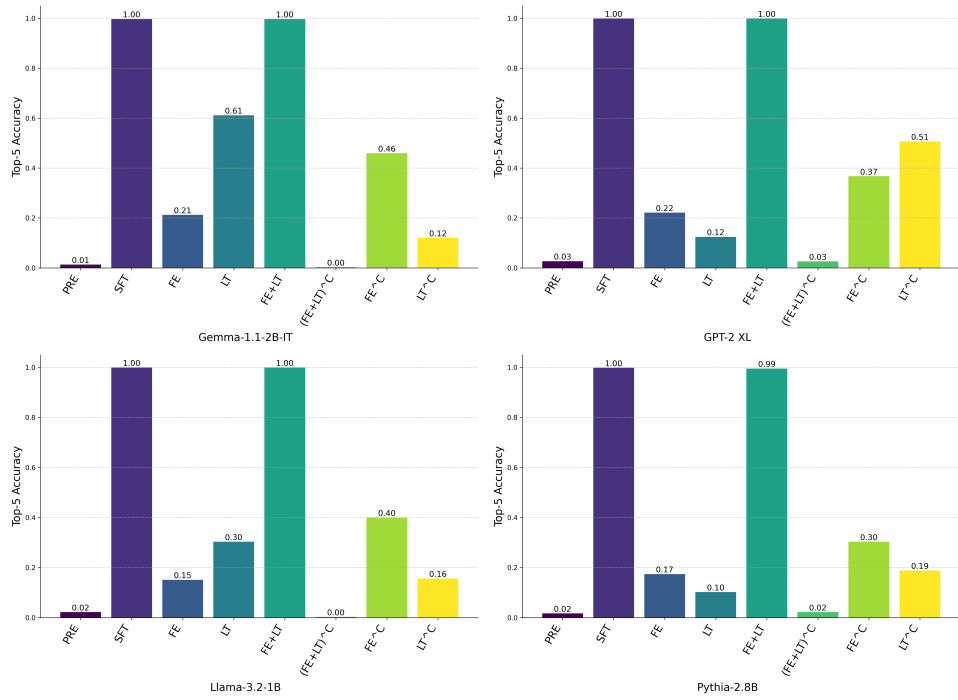


Figure 8: Top-5 accuracy under position grafting on Sentence 2 (QA) for the Fake Movies, Real Actors dataset, across the same four models.

C.1.2 TOKEN RANK RESULTS FOR QA EXAMPLES

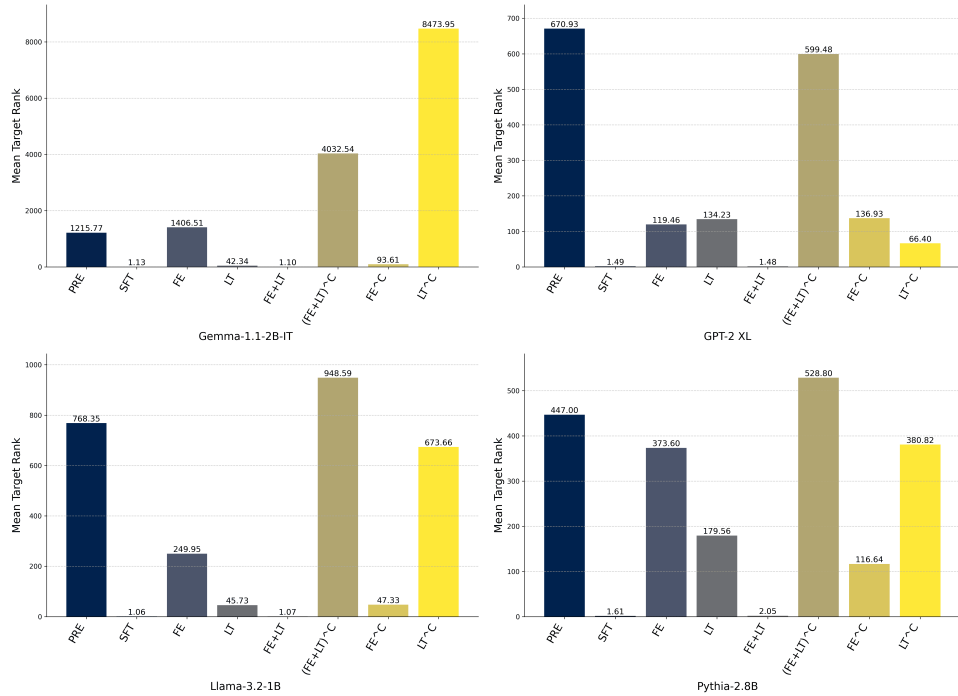


Figure 9: Mean rank of the target token under position grafting on Sentence 1 (headline) for the Fake Movies, Real Actors dataset; lower is better.

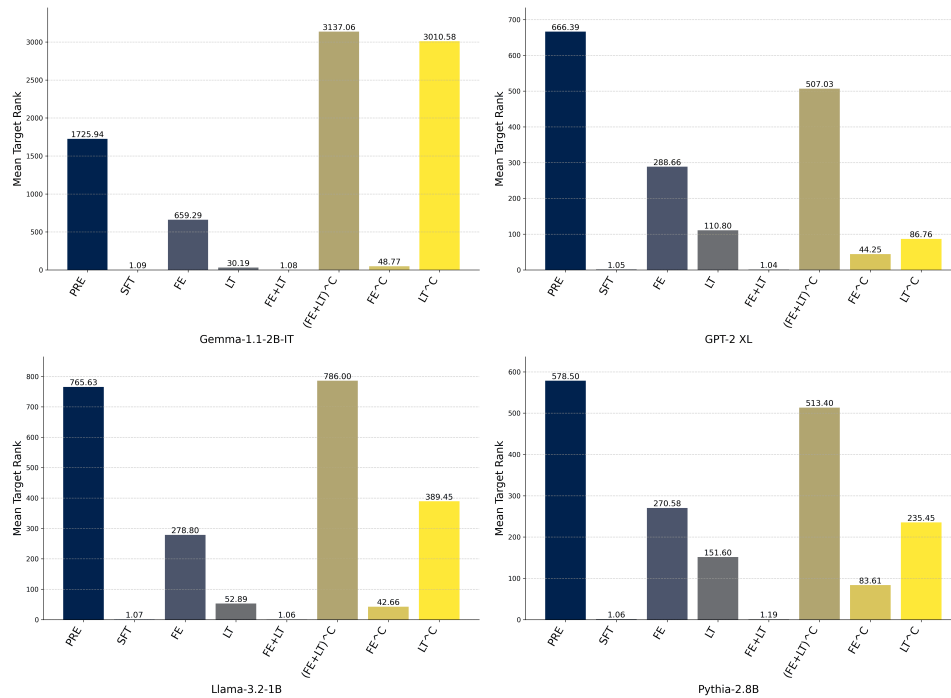


Figure 10: Mean rank of the target token under position grafting on Sentence 2 (QA) for the Fake Movies, Real Actors dataset; lower is better.

C.2 ADDITIONAL TOP-K RESULTS

We further vary k on the Fake Movies, Real Actors dataset. The qualitative pattern is invariant to this choice; only the strength of the individual “enrichment” and “recall” pathways diminishes as k decreases.

C.2.1 TOP-1 RESULTS FOR SENTENCE 1

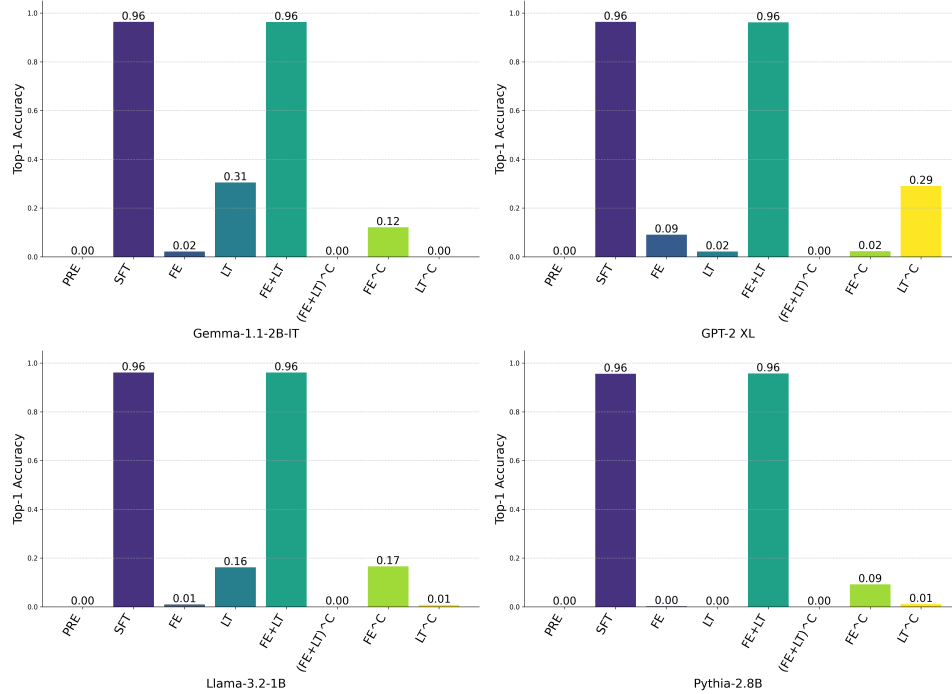


Figure 11: Top-1 accuracy under position grafting on Sentence 1 for the Fake Movies, Real Actors dataset, across all four models.

C.2.2 TOP-10 RESULTS

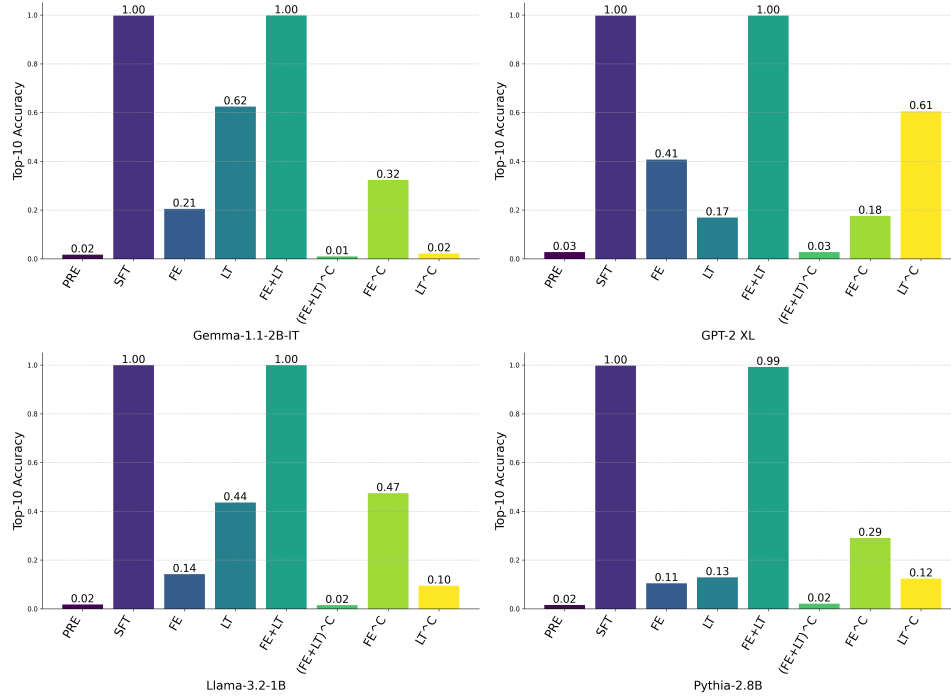


Figure 12: Top-10 accuracy under position grafting on Sentence 1 for the Fake Movies, Real Actors dataset, across all four models.

C.2.3 TOP-100 RESULTS

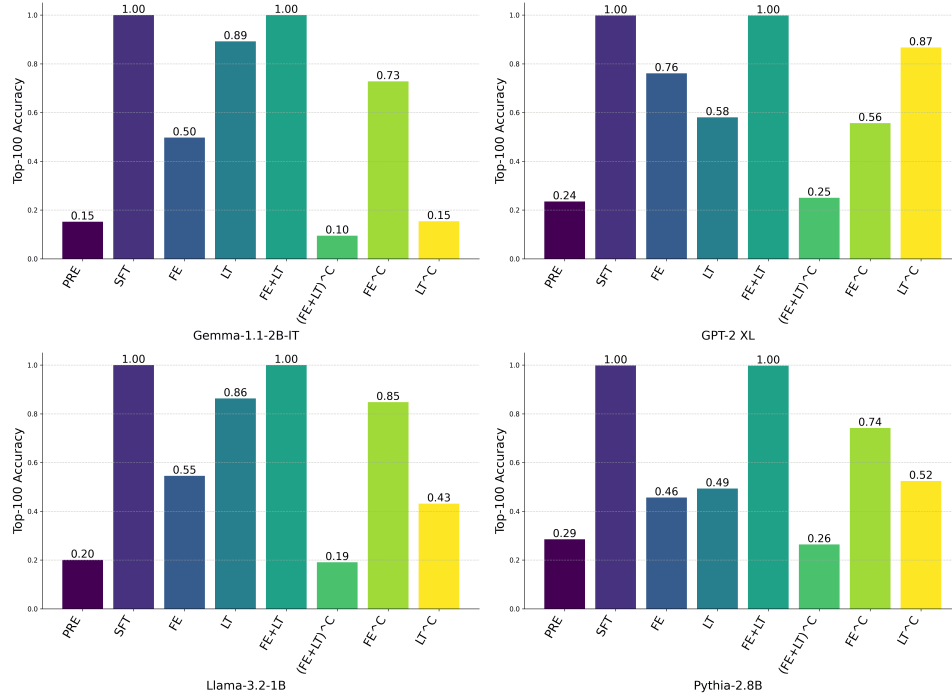


Figure 13: Top-100 accuracy under position grafting on Sentence 1 for the Fake Movies, Real Actors dataset, across all four models.

C.3 ADDITIONAL RESULTS FOR FAKE MOVIES, FAKE ACTORS

C.3.1 TOP-5 ACCURACY RESULTS FOR QA EXAMPLES

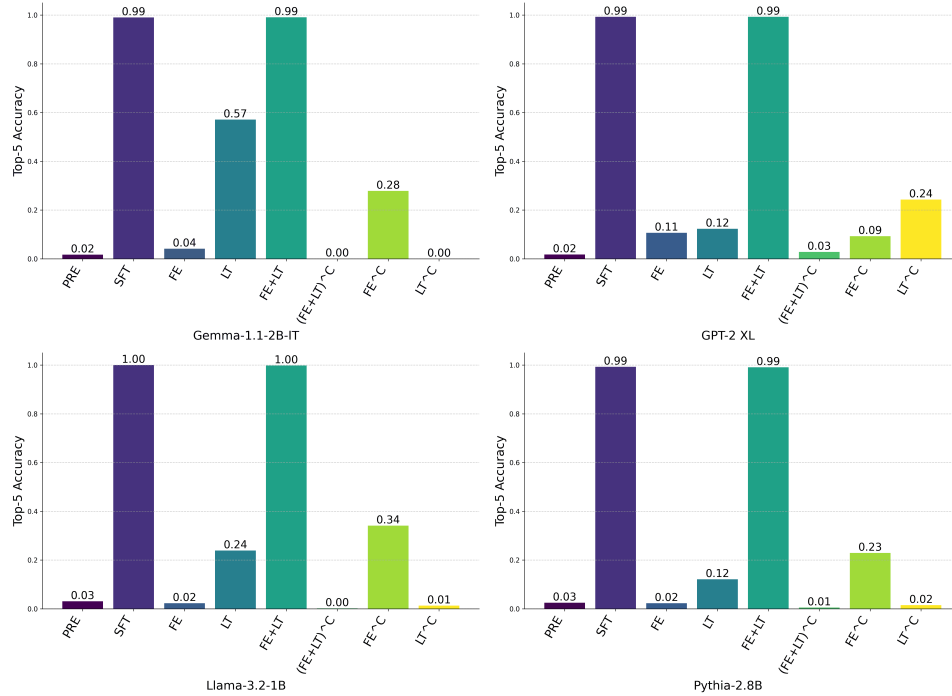


Figure 14: Top-5 accuracy under position grafting on Sentence 1 (headline) for the Fake Movies, Fake Actors dataset, across all four models.

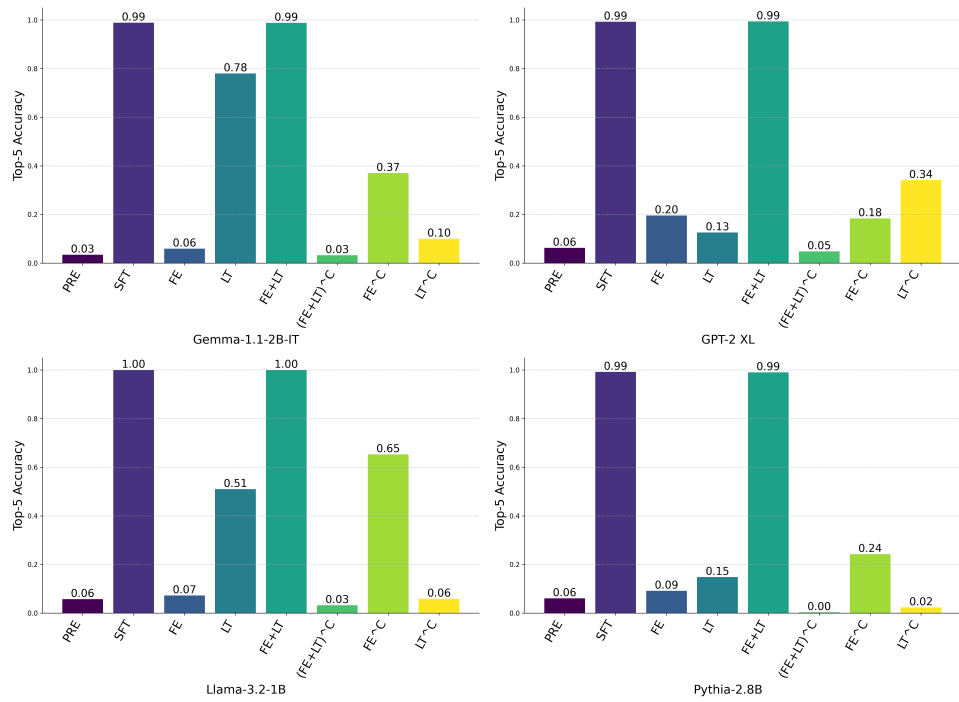


Figure 15: Top-5 accuracy under position grafting on Sentence 2 (QA) for the Fake Movies, Fake Actors dataset, across all four models.

C.4 ADDITIONAL RESULTS FOR REAL MOVIES, REAL ACTORS (SHUFFLED)

C.4.1 TOP-5 ACCURACY RESULTS FOR QA EXAMPLES

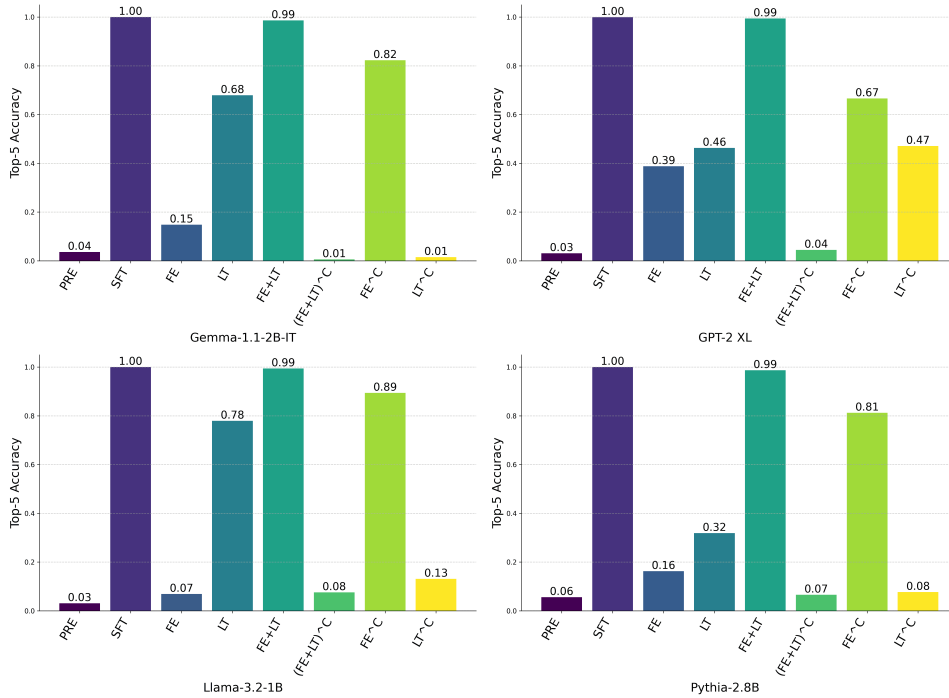


Figure 16: Top-5 accuracy under position grafting on Sentence 1 (headline) for the Real Movies, Real Actors (shuffled) dataset, across all four models.

C.5 MOVIE TITLE RESULTS

These results concern dynamic weight grafting when the movie title is included in the test sentence. The title alone does not suffice to recover the correct entity, whereas the title combined with the last token improves accuracy for every model. The interaction between the title and the first entity is inconsistent: relative to grafting the first entity alone, adding the title helps GPT-2 XL, leaves Llama 3 and Pythia essentially unchanged, and hurts Gemma. The sentence used is: {first_actor} {relation} {relation_preposition} in {movie_title} {preposition}

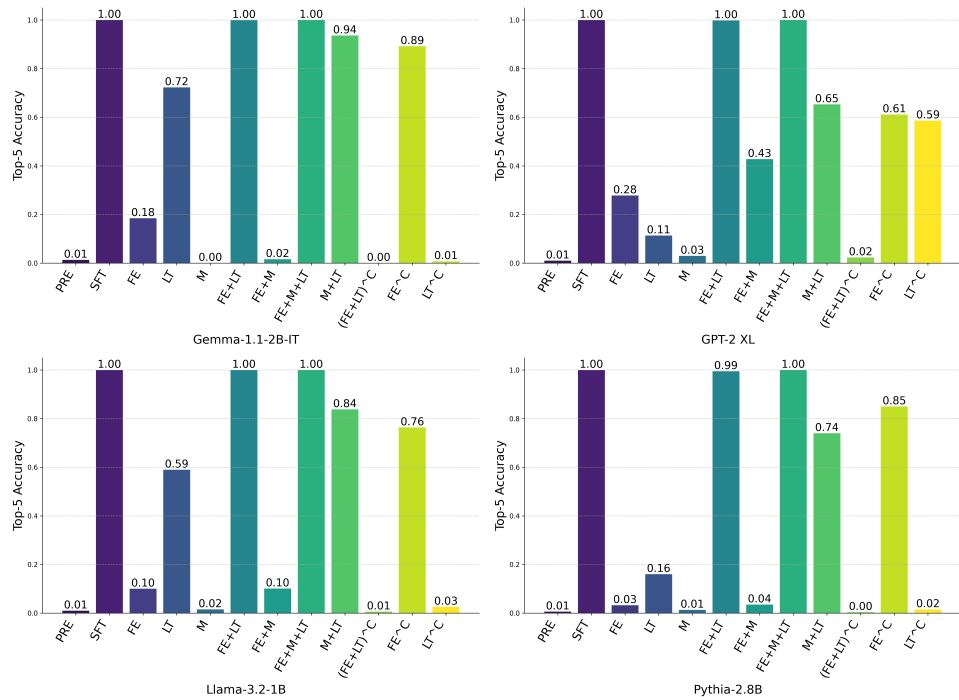


Figure 17: Top-5 accuracy on Sentence 3, which includes the movie title, for the Fake Movies, Real Actors dataset across all four models; “M” denotes grafting the movie title tokens.

C.6 UNEMBEDDING MATRIX RESULTS

These results use the finetuned unembeddings. Although top- k accuracy shifts somewhat, most visibly for single-token-position grafts, the overall pattern matches that obtained with the pretrained unembeddings.

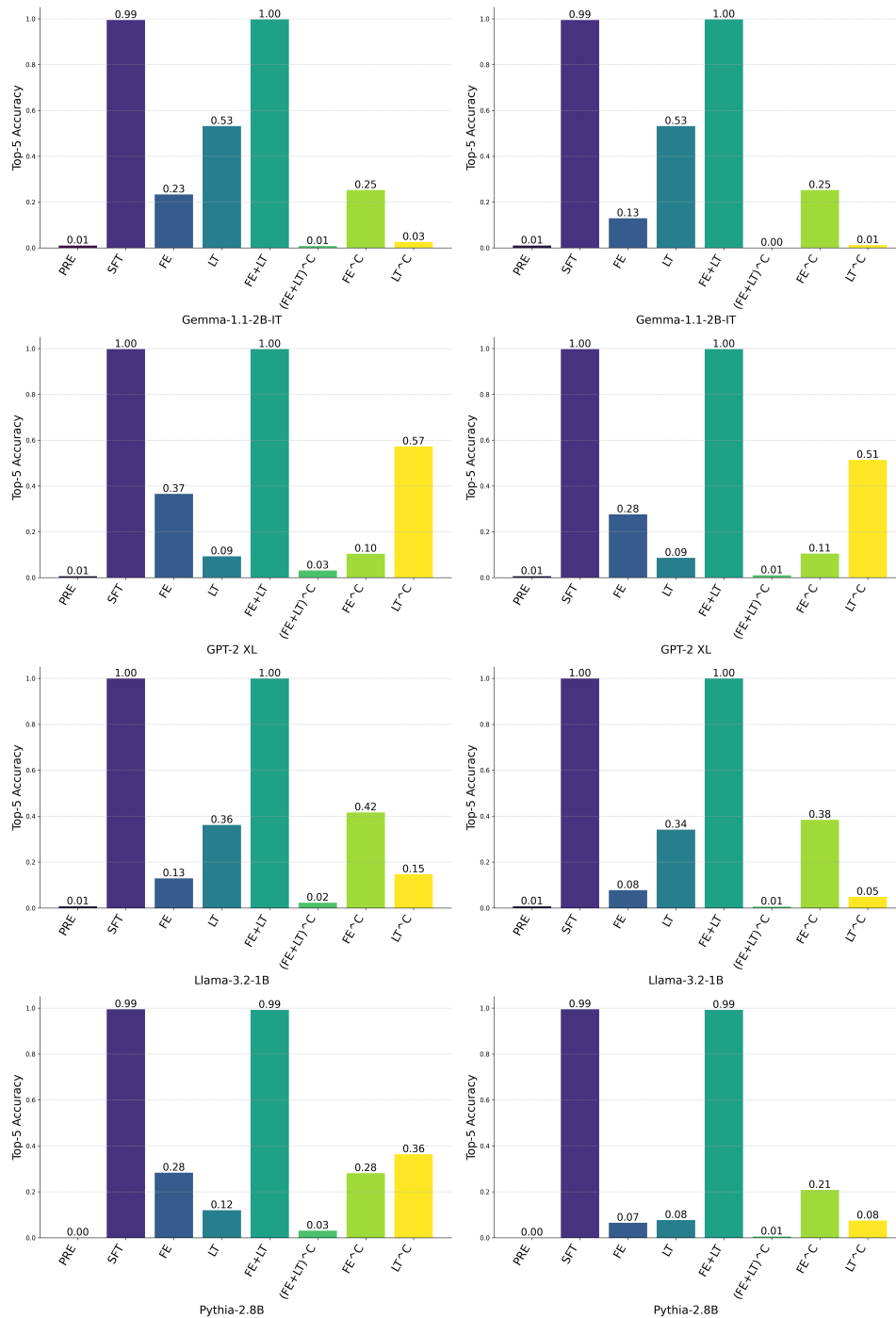


Figure 18: Top-5 accuracy on Sentence 1, comparing finetuned unembeddings (left) with pretrained unembeddings (right). The two choices yield the same overall pattern; the finetuned unembeddings raise top-5 accuracy only in the first-entity-only grafting condition.

C.7 LESS AGGRESSIVE FINETUNING RESULTS

This section reports the less aggressive finetuning experiments, which use a lower learning rate, zero weight decay, and supplemental training data from OpenWebText and IMDB. The pattern mirrors our other results, with the individual “extraction” and “recall” pathways somewhat weaker.

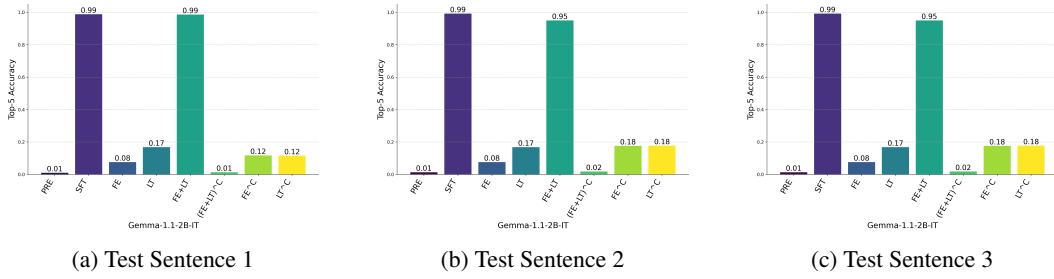


Figure 19: Top-5 accuracy for Gemma under less aggressive finetuning—a lower learning rate, zero weight decay, and supplemental training data from OpenWebText and IMDB—across the three test sentences.

C.8 COMPONENT-GRAFTING EXPERIMENT BASELINES

This section reports baseline results for the component-grafting experiments, grafting from the SFT model into the pretrained model.

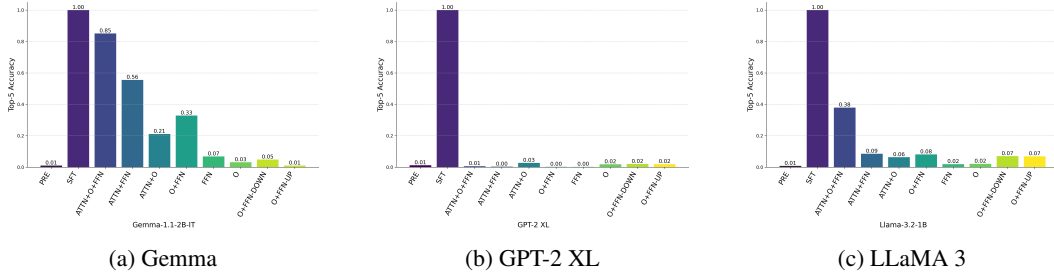


Figure 20: Top-5 accuracy for the component-grafting baselines on Sentence 1, grafting from SFT into the pretrained model, shown for Gemma, GPT-2 XL, and Llama 3.

C.9 REVERSAL CURSE COMPONENT-GRAFTING EXPERIMENT RESULTS

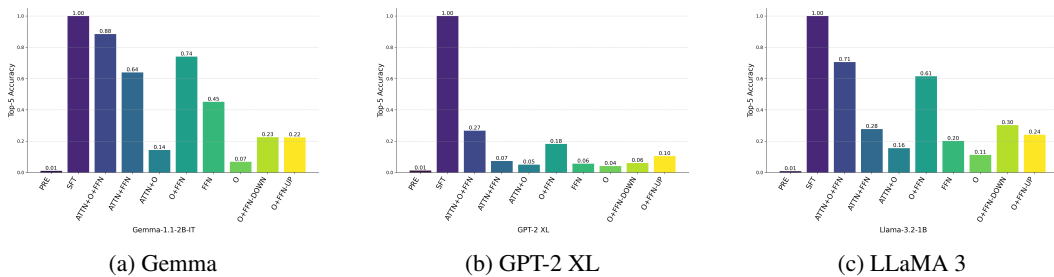


Figure 21: Top-5 accuracy for the reversal-curse component-grafting experiments on Sentence 1, shown for Gemma, GPT-2 XL, and Llama 3.

1728 C.10 TOKEN PROBABILITIES

1729
1730 We present five randomly sampled token-probability results, each listing the top 10 tokens for a
1731 representative example. Although our evaluation is based on top-5 accuracy, we display the top 10
1732 tokens per example to provide additional context.
1733

1734 C.10.1 LLAMA3: FE^C

1735
1736 Example 1:
1737 Target: Carolyn: 0.002
1738 J: 0.113
1739 L: 0.041
1740 Ch: 0.040
1741 Julie: 0.034
1742 Nicole: 0.024
1743 Nick: 0.023
1744 Michael: 0.023
1745 Jon: 0.023
1746 Jamie: 0.021
1747 Kelly: 0.017

1748 -----
1749 Example 2:
1750 Target: Lindsay: 0.024
1751 Michael: 0.093
1752 Ashley: 0.061
1753 Jennifer: 0.038
1754 Kim: 0.032
1755 Alexander: 0.025
1756 Lindsay: 0.024
1757 Milo: 0.023
1758 L: 0.021
1759 Jason: 0.018
1760 Lisa: 0.018

1761 -----
1762 Example 3:
1763 Target: Gwen: 0.015
1764 Michael: 0.030
1765 Paul: 0.028
1766 Julie: 0.024
1767 Elizabeth: 0.020
1768 Mary: 0.020
1769 S: 0.019
1770 E: 0.018
1771 J: 0.017
1772 L: 0.017
1773 A: 0.017

1774 -----
1775 Example 4:
1776 Target: Dominic: 0.006
1777 Is: 0.039
1778 Sarah: 0.031
1779 Jason: 0.026
1780 D: 0.026
1781 Ellen: 0.025
Michael: 0.021
Jennifer: 0.020
Elizabeth: 0.020
Ann: 0.019

1782 Mark: 0.019
1783

1784 -----

1785 Example 5:

1786 Target: Ch: 0.045

1787 Ch: 0.045

1788 David: 0.024

1789 John: 0.022

1790 Peter: 0.018

1791 L: 0.018

1792 Mark: 0.018

1793 Michael: 0.017

1794 Richard: 0.016

1795 Ben: 0.012

1796 James: 0.012
1797

1798 -----

1799

1800 C.10.2 GEMMA: LT

1801 Example 1:

1802 Target: Elizabeth: 0.040

1803 Julie: 0.495

1804 John: 0.130

1805 Stephen: 0.076

1806 Elizabeth: 0.040

1807 Hilary: 0.021

1808 Laurel: 0.018

1809 Tyne: 0.017

1810 Marian: 0.014

1811 Victoria: 0.013

1812 Juliet: 0.008
1813

1814 -----

1815 Example 2:

1816 Target: Uta: 0.008

1817 John: 0.901

1818 Joan: 0.022

1819 Sally: 0.011

1820 Chelsea: 0.009

1821 Uta: 0.008

1822 Tyne: 0.003

1823 Gina: 0.002

1824 Elle: 0.001

1825 Lisa: 0.001

1826 Rosemary: 0.001
1827

1828 -----

1829 Example 3:

1830 Target: Jennifer: 0.706

1831 Jennifer: 0.706

1832 John: 0.094

1833 Il: 0.028

1834 Me: 0.020

1835 Stephen: 0.015

1836 Carol: 0.012

1837 S: 0.008

1838 David: 0.008

1839 Elle: 0.007

1840 T: 0.006

```

1836 -----
1837
1838 Example 4:
1839 Target: Marcia: 0.017
1840 John: 0.152
1841 Gwen: 0.136
1842 Susan: 0.050
1843 Tim: 0.045
1844 Ch: 0.041
1845 Celeste: 0.037
1846 Tara: 0.030
1847 T: 0.028
1848 Elle: 0.028
1849 Brittany: 0.026
1850 -----

```

```

1851 Example 5:
1852 Target: Heather: 0.035
1853 Ly: 0.916
1854 Heather: 0.035
1855 Hilary: 0.011
1856 Stephen: 0.007
1857 Julie: 0.002
1858 Rosemary: 0.002
1859 Ch: 0.002
1860 Wanda: 0.001
1861 Chelsea: 0.001
1862 Tem: 0.001
1863 -----

```

1864 C.10.3 GPT-2 XL: (FE+LT)^C

```

1865 Example 1:
1866 Target: Fre: 0.000
1867 her: 0.037
1868 John: 0.018
1869 Jason: 0.017
1870 Chris: 0.016
1871 Adam: 0.015
1872 Zach: 0.014
1873 Josh: 0.013
1874 the: 0.013
1875 Michael: 0.013
1876 Ben: 0.012
1877 -----

```

```

1878 Example 2:
1879 Target: A: 0.002
1880 her: 0.043
1881 the: 0.017
1882 John: 0.016
1883 Tom: 0.016
1884 Peter: 0.012
1885 fellow: 0.012
1886 Michael: 0.011
1887 David: 0.011
1888 James: 0.011
1889 Jack: 0.010
1890 -----

```

```
1890 Example 3:
1891 Target: Matthew: 0.003
1892 his: 0.057
1893 the: 0.017
1894 Jennifer: 0.014
1895 fellow: 0.012
1896 Chris: 0.011
1897 Michael: 0.011
1898 Jason: 0.009
1899 John: 0.008
1900 James: 0.008
1901 Jessica: 0.007
```

```
1902 -----
1903 Example 4:
1904 Target: Cher: 0.000
1905 her: 0.042
1906 Tom: 0.026
1907 Robert: 0.019
1908 the: 0.018
1909 Matt: 0.018
1910 Michael: 0.016
1911 Brad: 0.016
1912 Chris: 0.014
1913 Johnny: 0.014
1914 John: 0.014
```

```
1915 -----
1916 Example 5:
1917 Target: Timothy: 0.001
1918 his: 0.056
1919 the: 0.025
1920 Michael: 0.013
1921 fellow: 0.013
1922 Robert: 0.013
1923 John: 0.012
1924 James: 0.010
1925 Tom: 0.010
1926 Mark: 0.008
1927 Peter: 0.008
1928 -----
```

1929 C.10.4 PROBABILITY ON COMMON TOKENS WITH TARGET IN TOP 5

1930 The following is a characteristic result—for Gemma grafted on the first entity token—in which the
1931 model concentrates high probability on common tokens while still ranking the target token within the
1932 top 5.

```
1933 Target: Robert: 0.016
1934 a: 0.365
1935 an: 0.151
1936 the: 0.091
1937 Robert: 0.016
1938 two: 0.009
1939 his: 0.008
1940 other: 0.006
1941 John: 0.005
1942 Morgan: 0.005
1943 my: 0.005
```